# Machine Learning

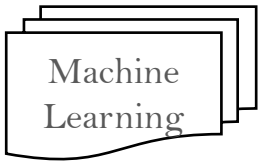
## Lecture 11: Large Language Models

**Spring 2025**

**Instructor: Xiaodong Gu**

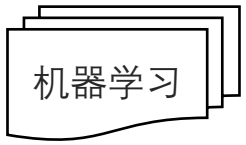


# When Transformer Meets Insufficient Data




English Text

Transformer




Chinese Text

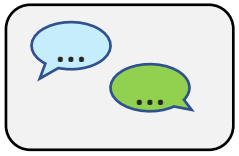


Documents

Transformer



Summary



Question

Transformer



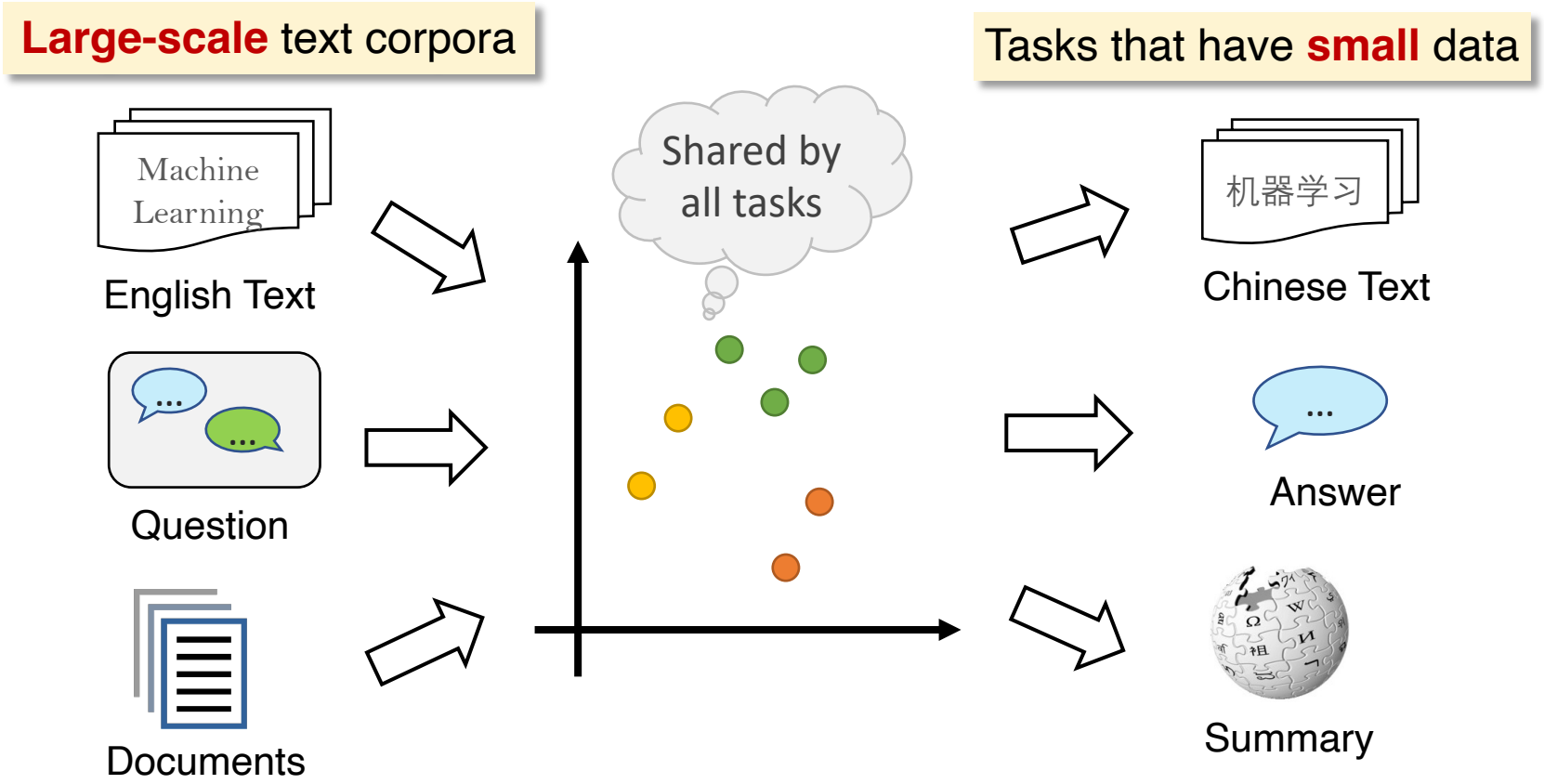
Ansewr

⋮

Any Problems
So Far?

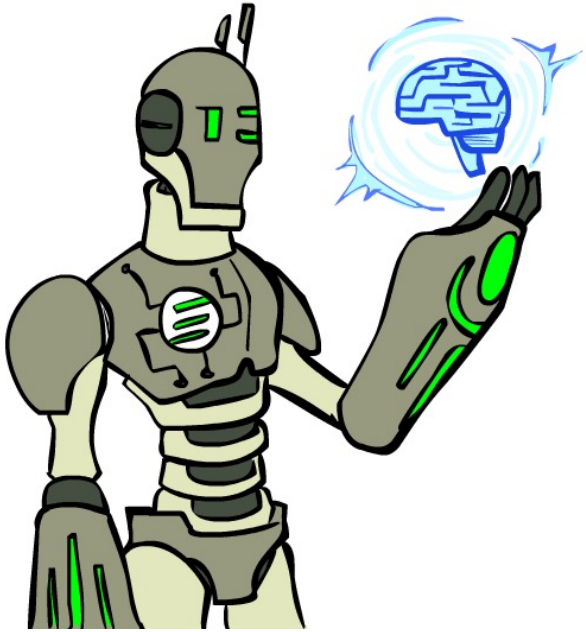
# Thinking…

## Can we learn a unified vector space for all texts?



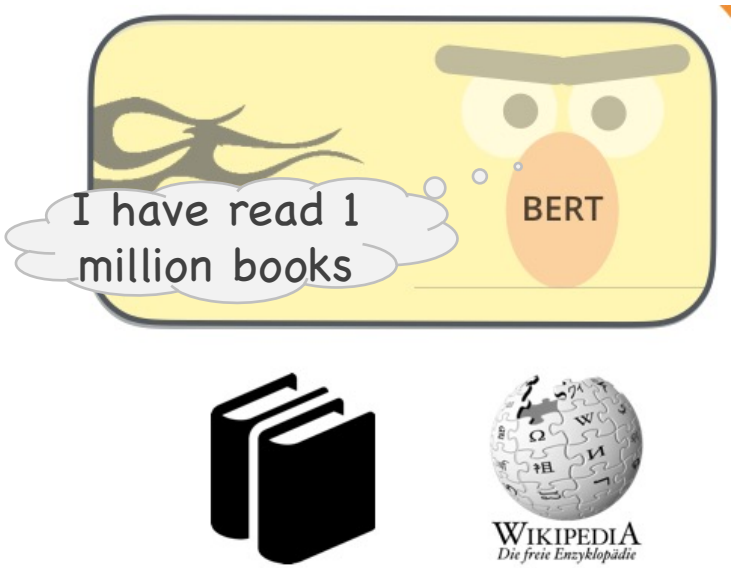
# Today

- Pre-trained Language Models
- Large Language Models



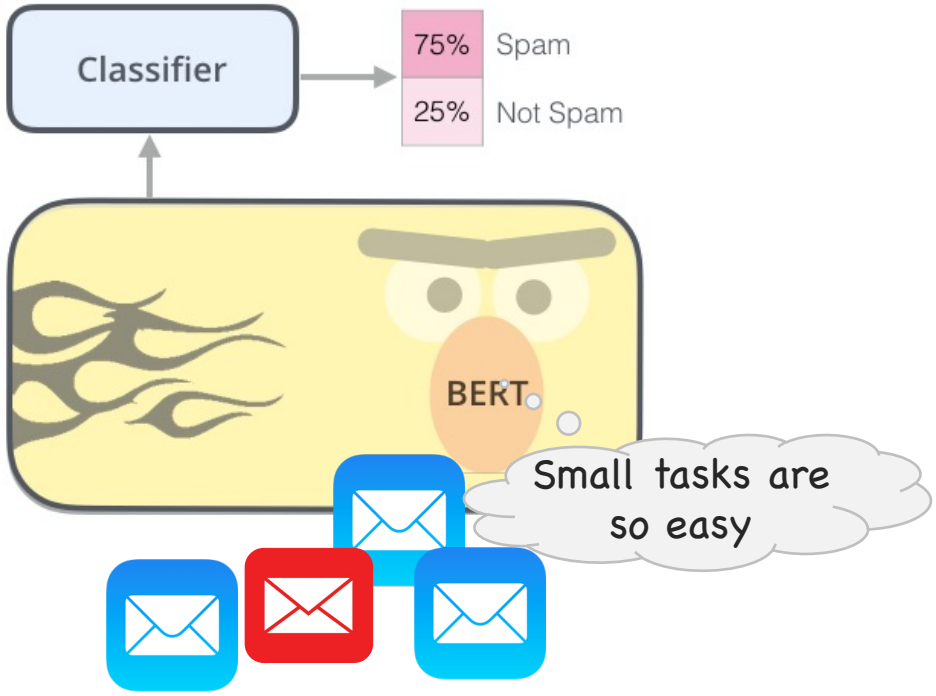
# Pre-trained Language Models

Unsupervised training on large amounts of text (e.g., books, Wikipedia, etc)




**Phase 1: Pre-Training**

Supervised training on a specific task with a labeled dataset. (e.g., spam detection)



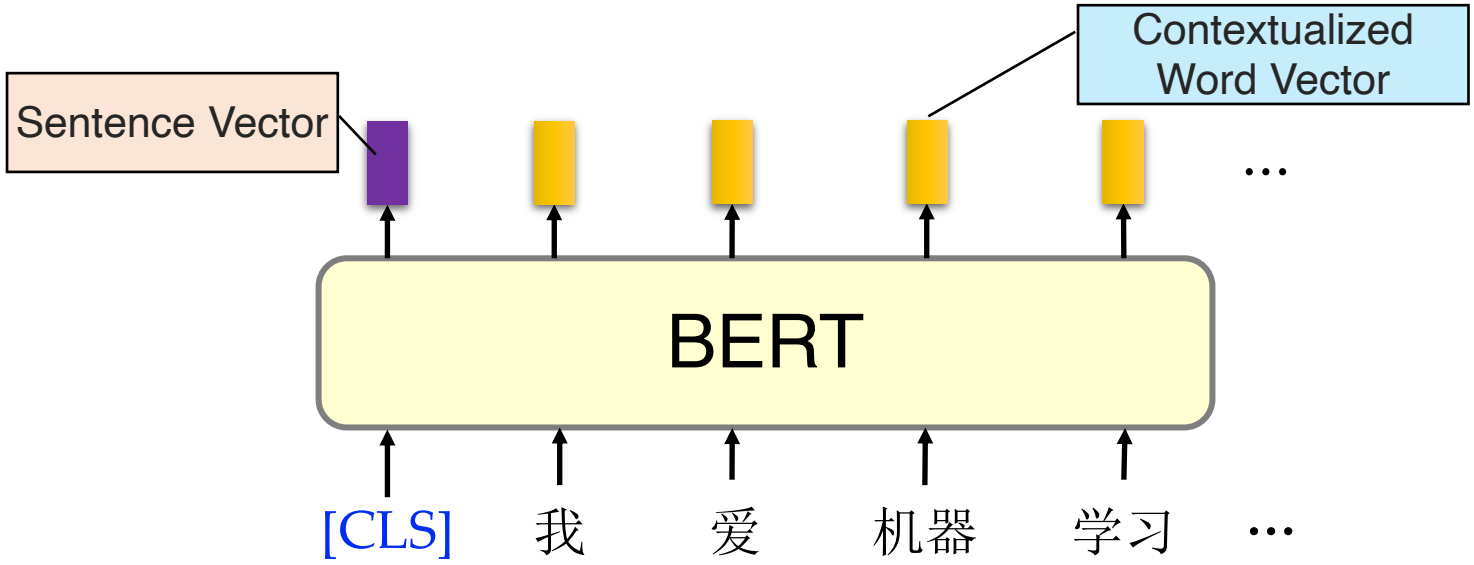

**Phase 2: Fine-Tuning**

# BERT – Bidirectional Encoder Representations from Transformers



## A Transformer Encoder that

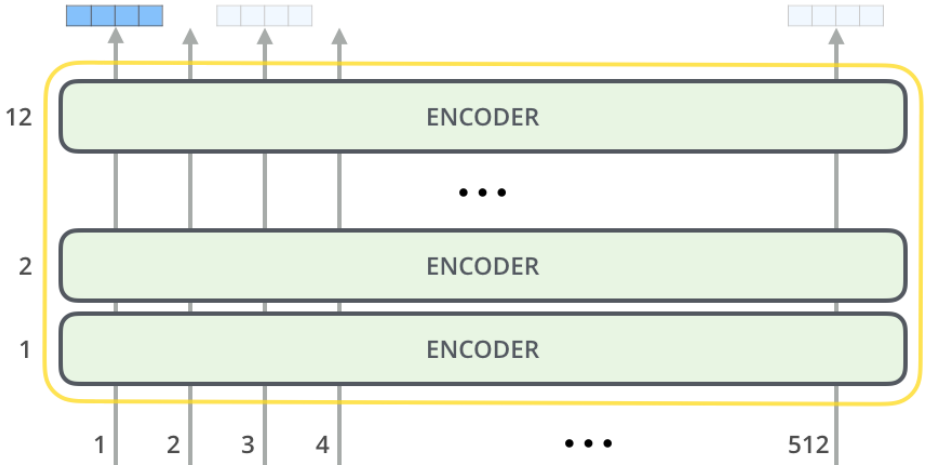
- allows for learning representations of words and sentences.
- pre-trained on large-scale text corpora and then;
- fine-tuned on small task-specific datasets (e.g., classification, QA.)



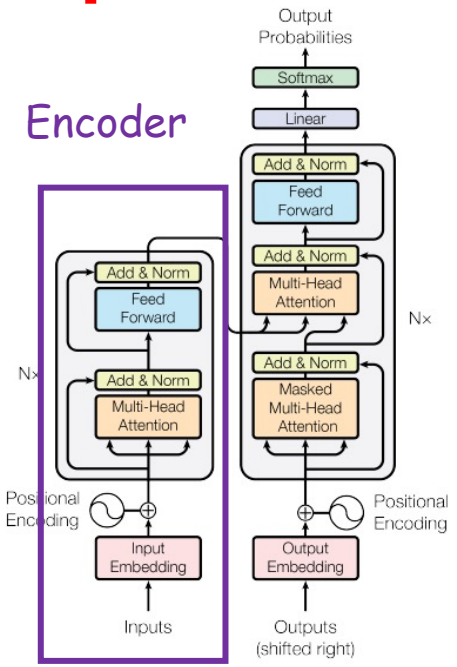
# Model Architecture

BERT = Transformer Encoder

- BERT takes a sequence of words as input which keep flowing up the stack.
- The **first** input token is always a special [CLS] token which stands for the classification.

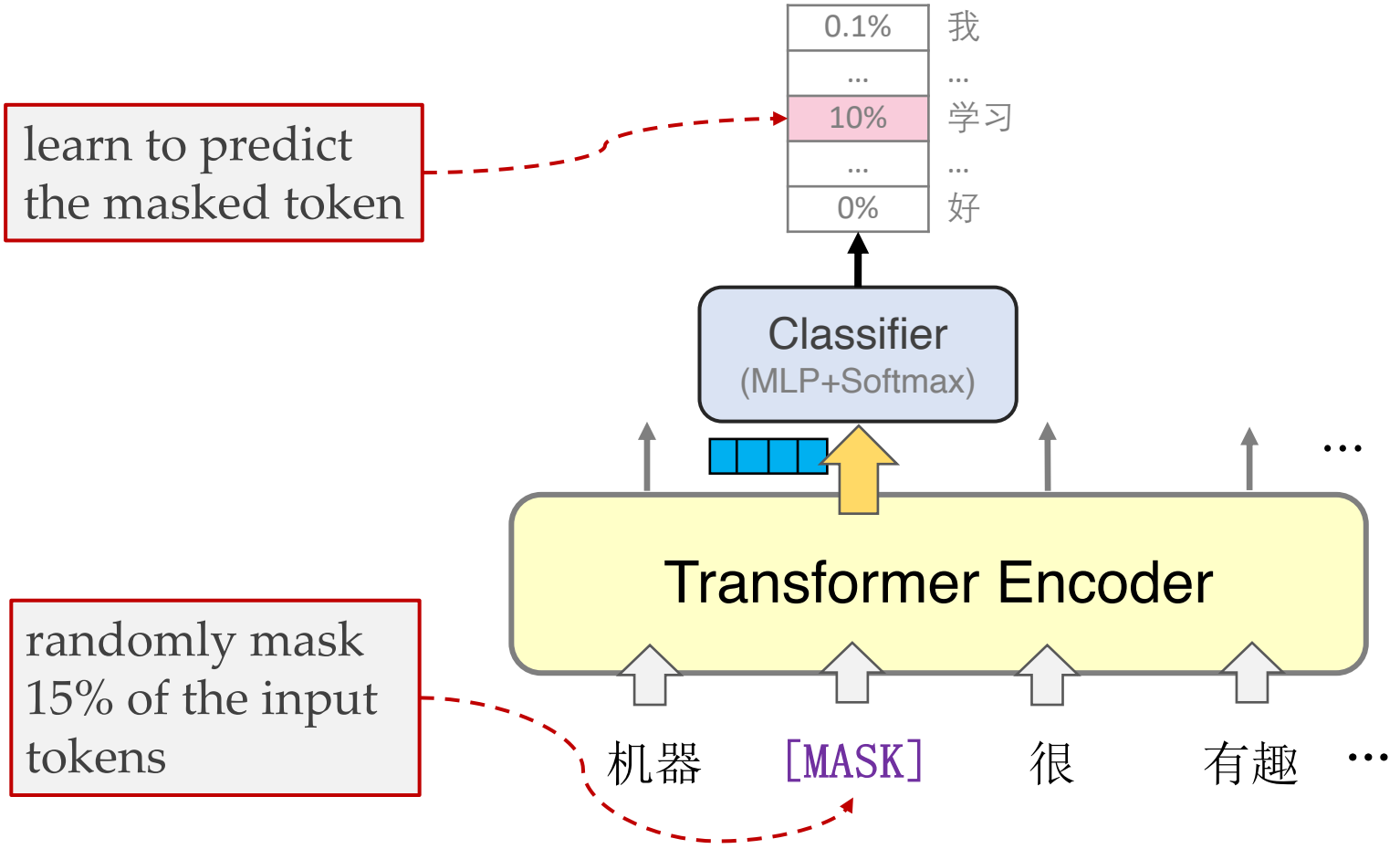



=



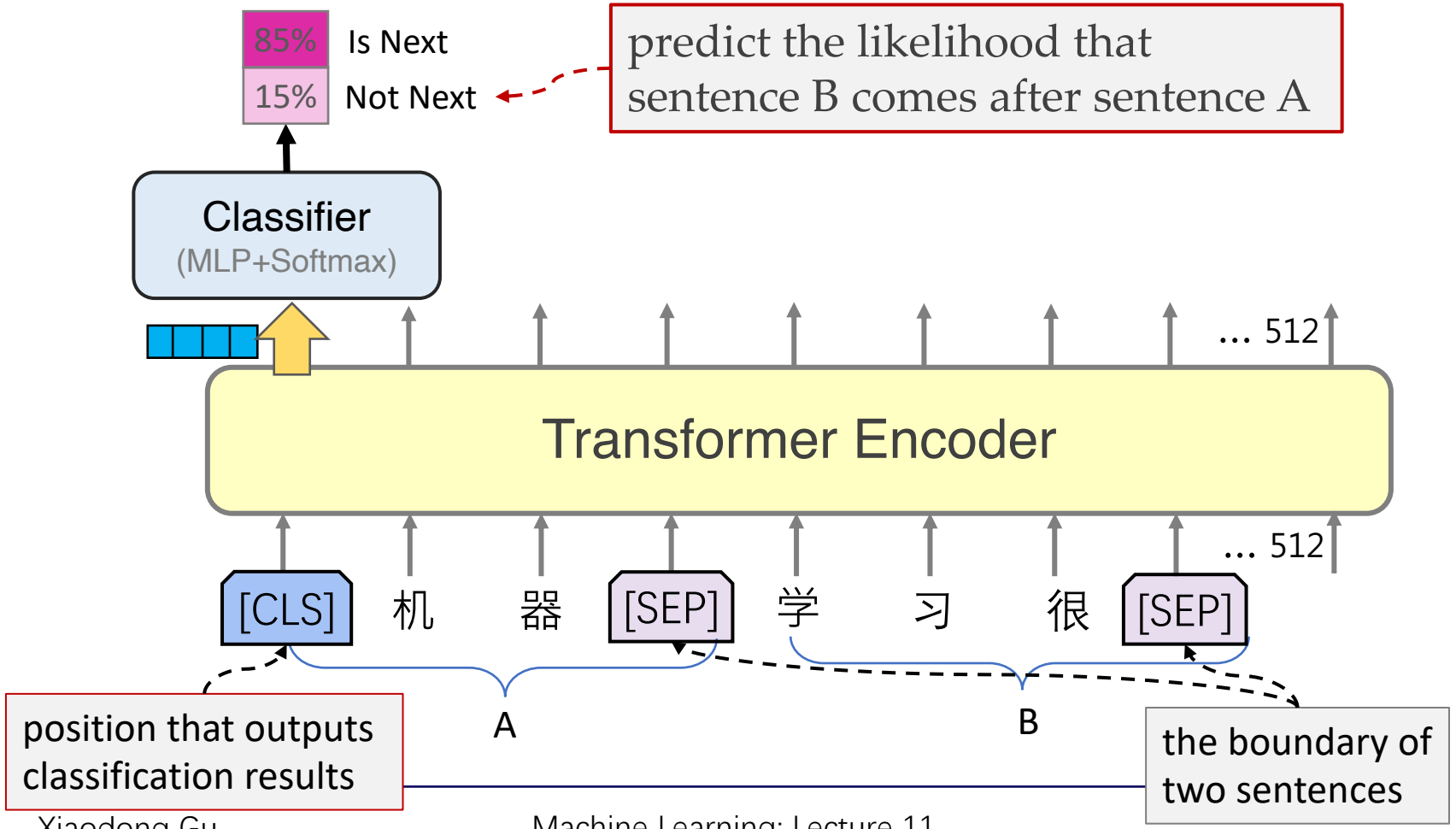
# Pre-Training

Task 1: Masked Language Model (MLM)



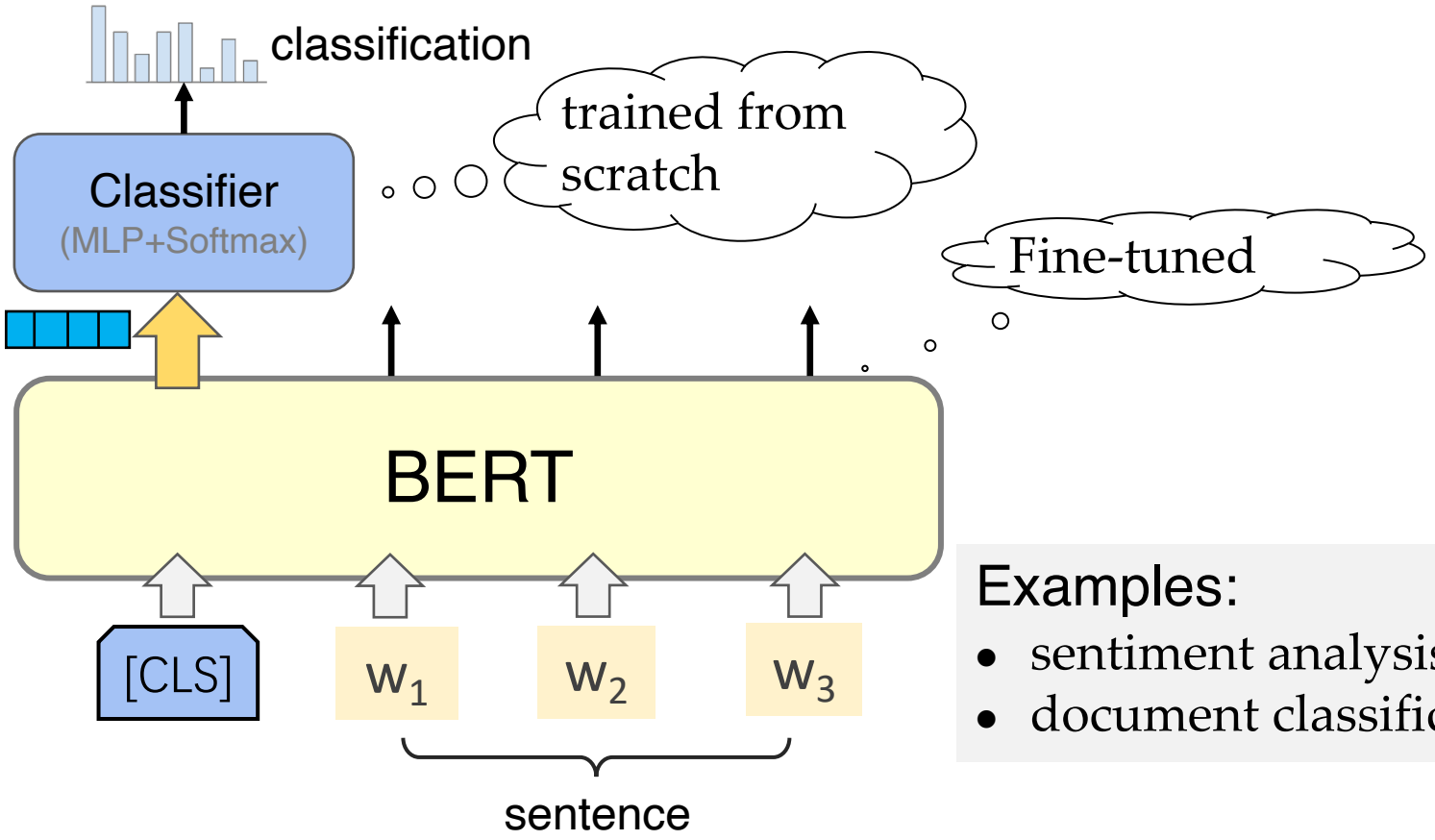
# Pre-Training

## Task 2: Next Sentence Prediction (NSP)



# Fine-tuning: Sentence Classification

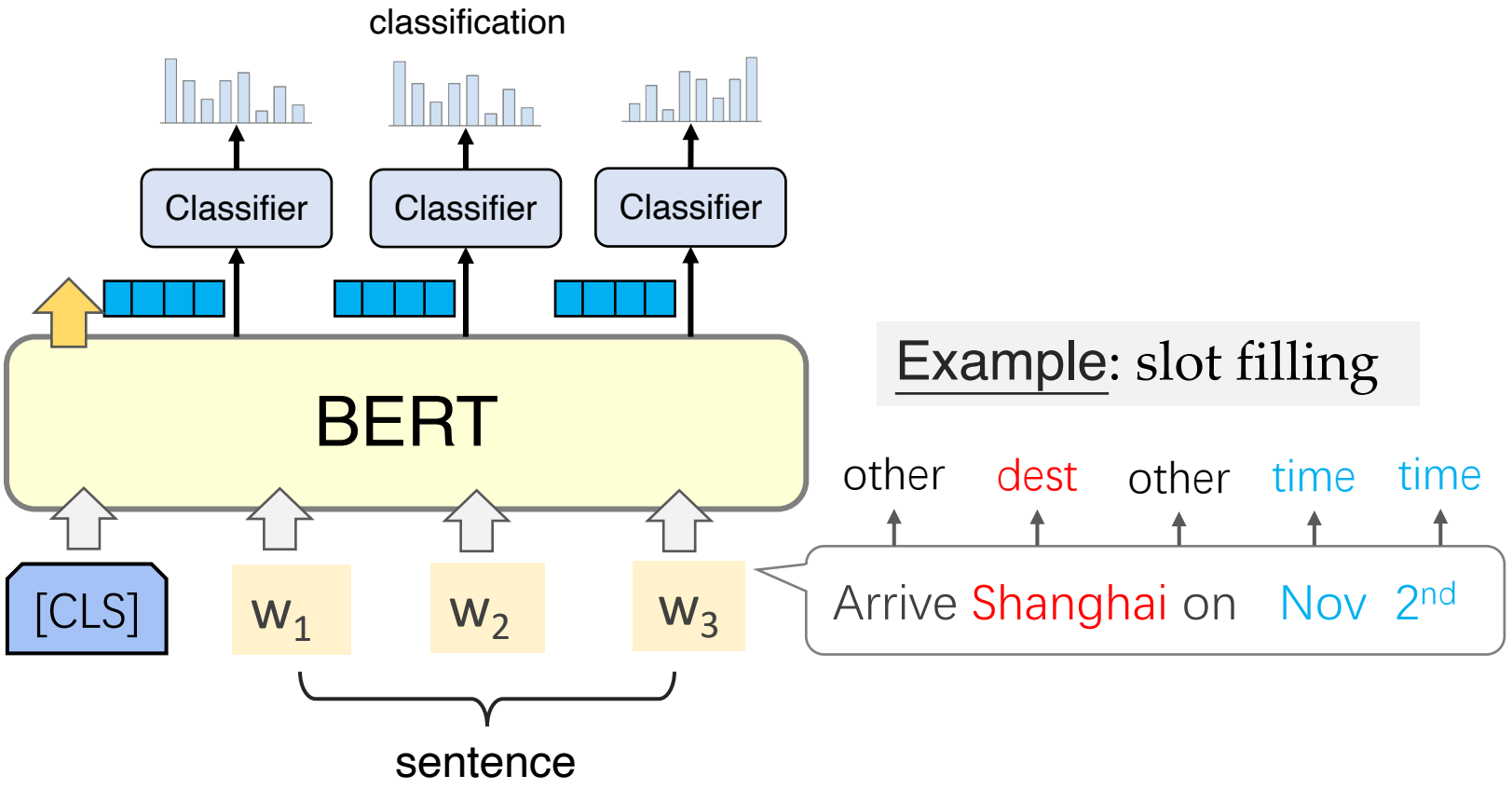
- **Input**: a single sentence, **Output**: sentence class




Examples:
- sentiment analysis
- document classification

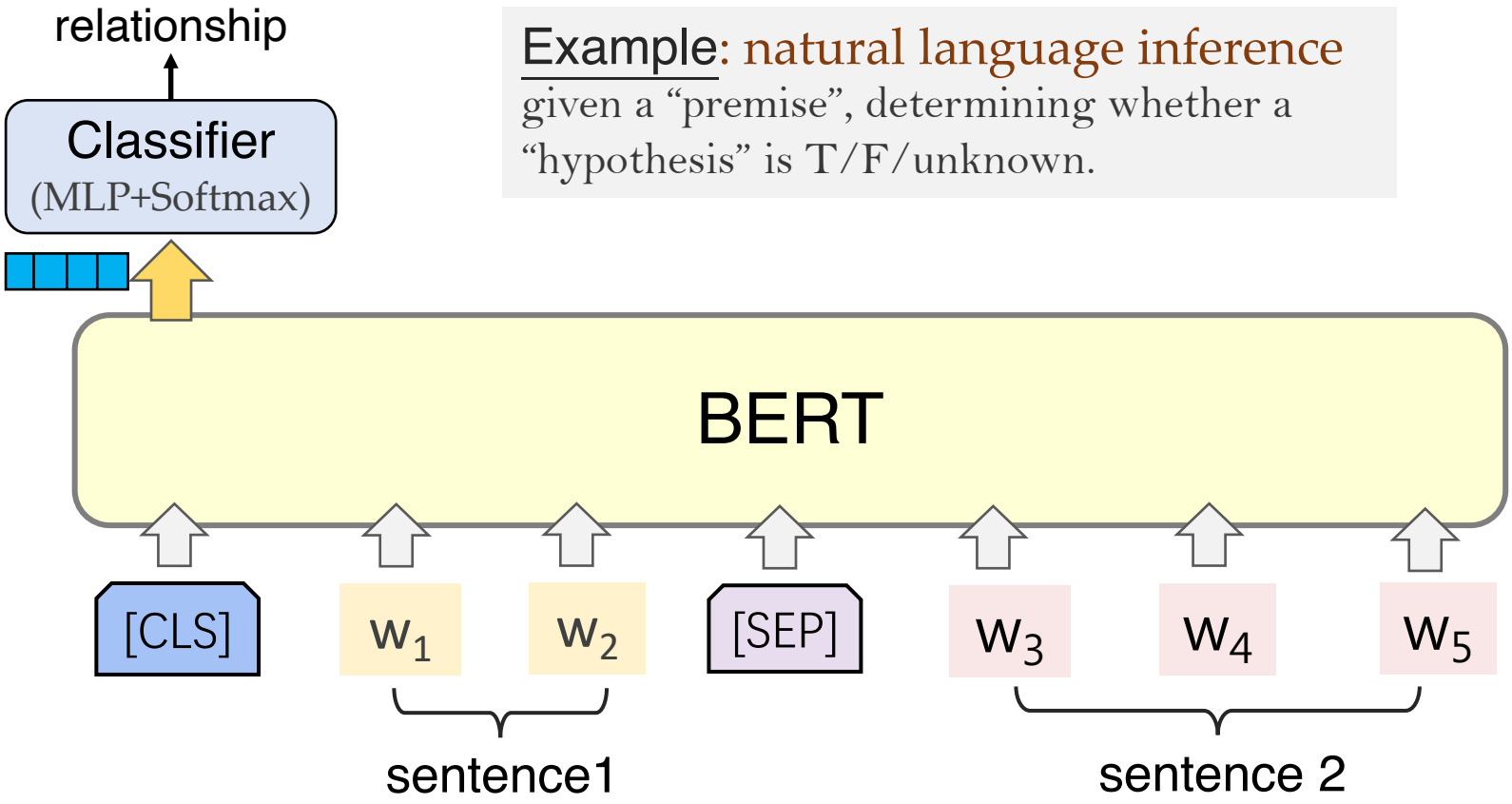
# Finetuning – Word Tagging

Input: a single sentence  Output: classification of each word



# Finetuning – Classifying Sentence Pairs

**input**: two sentences **output**: relationship




Example: natural language inference
given a "premise", determining whether a "hypothesis" is T/F/unknown.

# Finetuning – QA (Reading Comprehension)

**Task:** for a given question, find the answer in a paragraph.

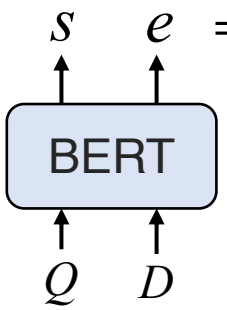
In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation 17 include drizzle, rain, sleet, snow, **graupel** and hail ... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in 77 scattered locations 79 are called "showers".

Q: what causes precipitations to fall?

A: gravity (s=17, e=17)

Q: where do water droplets collide with ice crystals to form precipitations?

A: within a cloud (s=77, e=79)

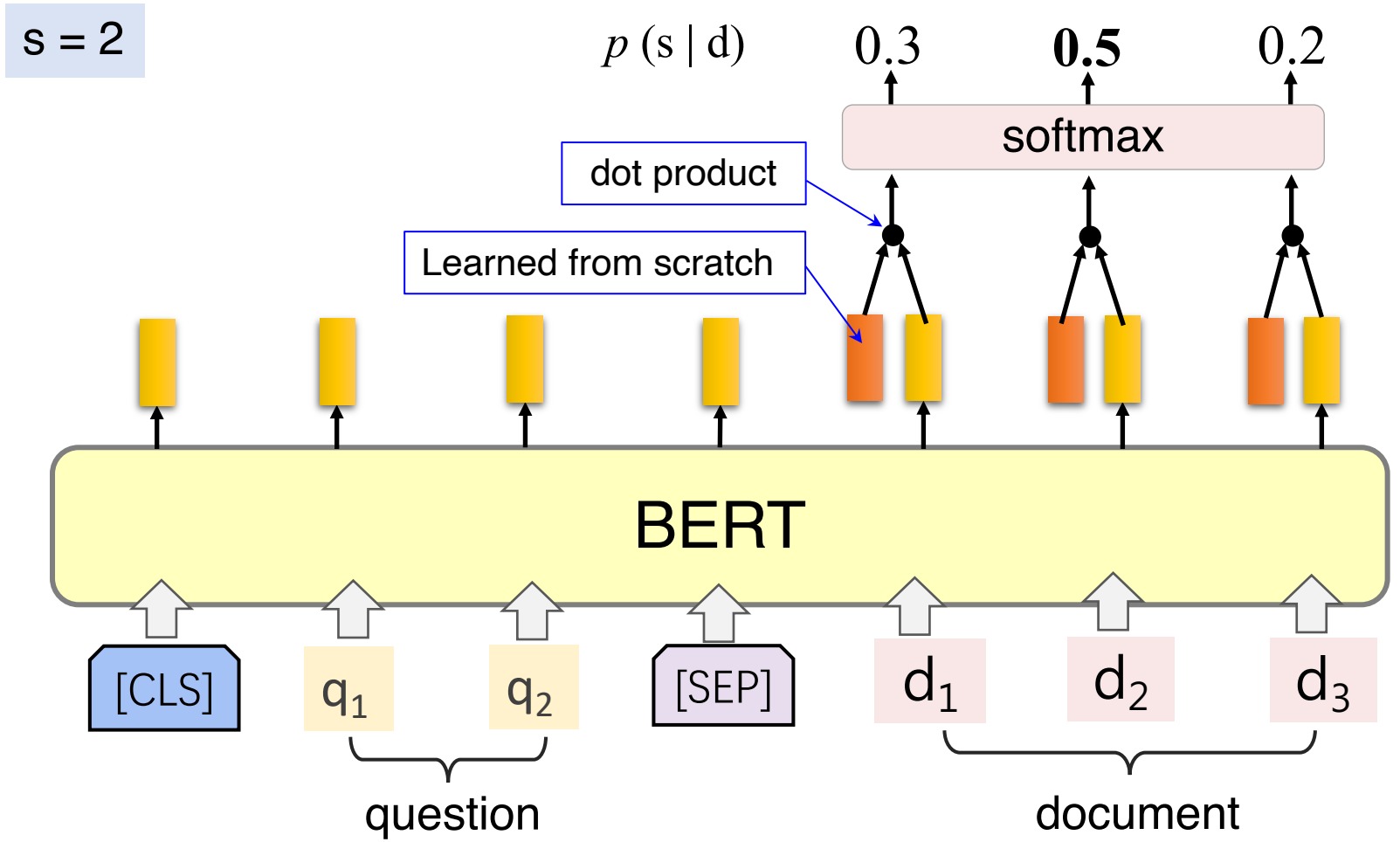



⇒ answer A = $\{d_s, \ldots, d_e\}$

**Input:**
a document D=$\{d_1, \ldots, d_N\}$
a question Q=$\{q_1, \ldots, q_N\}$

**Output:** two integers ($s$, $e$) which indicate the start and the end positions of the answer in the document.

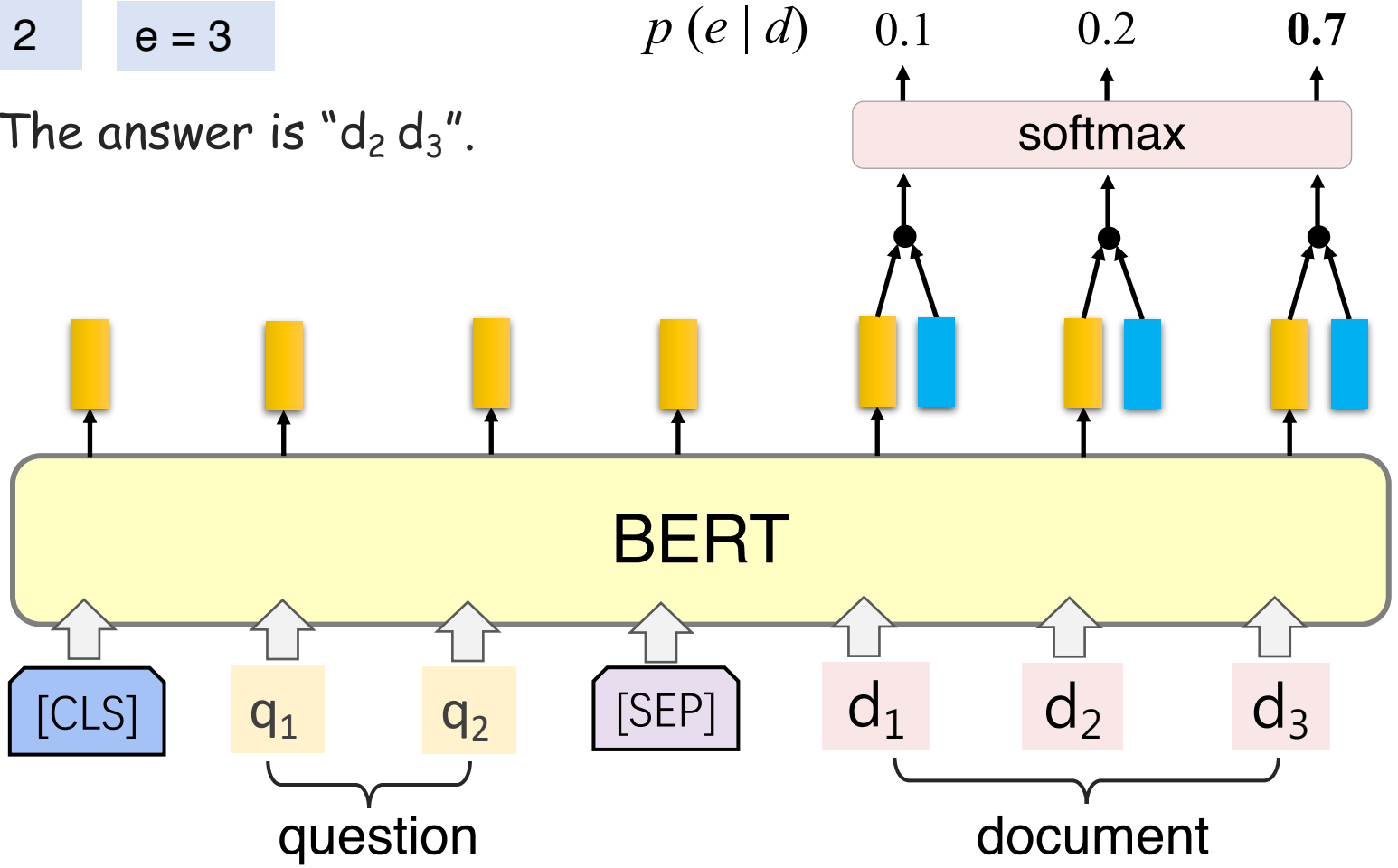
# Finetuning – QA (Reading Comprehension)



# Finetuning – QA (Reading Comprehension)

s = 2 e = 3

⇒ The answer is "$d_2$ $d_3$".



# BERT屠榜

## SQuAD 2.0

| Rank | Model | EM | F1 |
|---|---|---|---|
| | Human Performance<br>*Stanford University*<br>(Rajpurkar & Jia et al. '18) | 86.831 | 89.452 |
| 1<br>Mar 20, 2019 | BERT + DAE + AoA (ensemble)<br>*Joint Laboratory of HIT and iFLYTEK Research* | **87.147** | **89.474** |
| 2<br>Mar 15, 2019 | BERT + ConvLSTM + MTL + Verifier (ensemble)<br>*Layer 6 AI* | 86.730 | 89.286 |
| 3<br>Mar 05, 2019 | BERT + N-Gram Masking + Synthetic Self-Training (ensemble)<br>*Google AI Language*<br>https://github.com/google-research/bert | 86.673 | 89.147 |
| 4<br>May 21, 2019 | XLNet (single model)<br>*XLNet Team* | 86.346 | 89.133 |
| 5<br>Apr 13, 2019 | SemBERT(ensemble)<br>*Shanghai Jiao Tong University* | 86.166 | 88.886 |

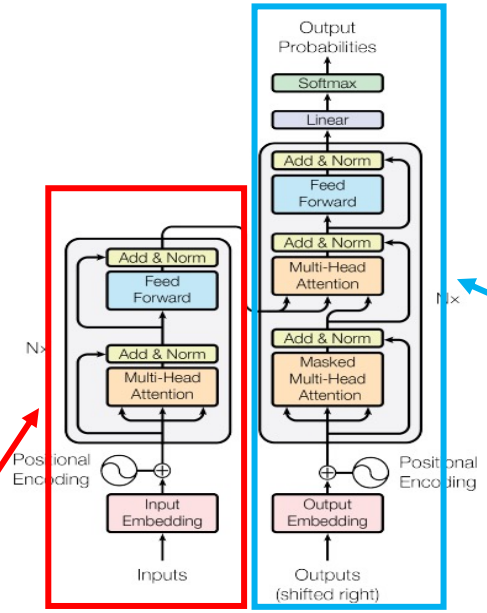
https://paperswithcode.com/sota/question-answering-on-squad20

# Generative Pre-Trained Transformer (GPT)

- A **pretrained language model** based on Transformer decoder.

BERT (340M)






GPT-2 (1542M)



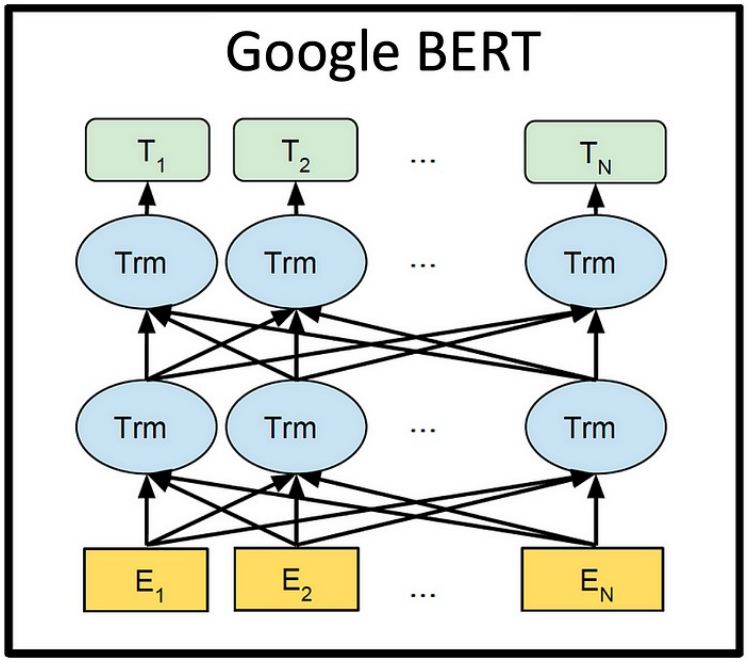
[OpenAI 2019]

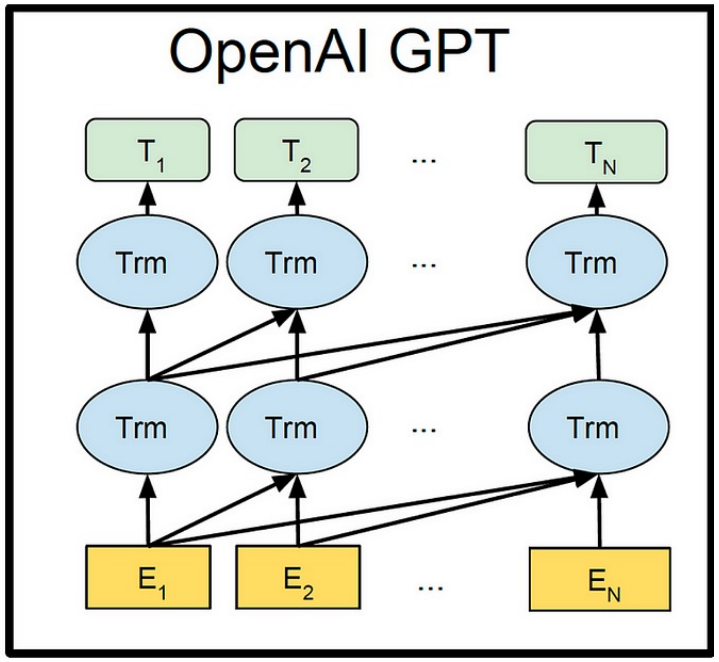
GPT-3 (175B)




[OpenAI 2020]

# BERT vs GPT

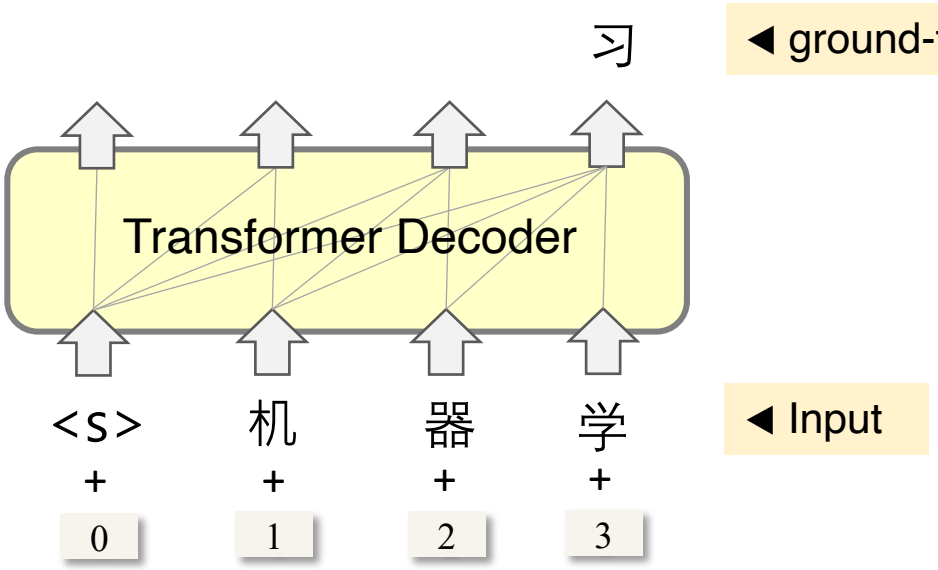

Google BERT
T1
T2
...
TN
Trm
Trm
...
Trm
Trm
Trm
...
Trm
E1
E2
...
EN


OpenAI GPT
T1
T2
...
TN
Trm
Trm
...
Trm
Trm
Trm
...
Trm
E1
E2
...
EN

# Pre-training GPT

**Simply language model:**

— Given the preceding n-1 tokens, predict the n-th token.

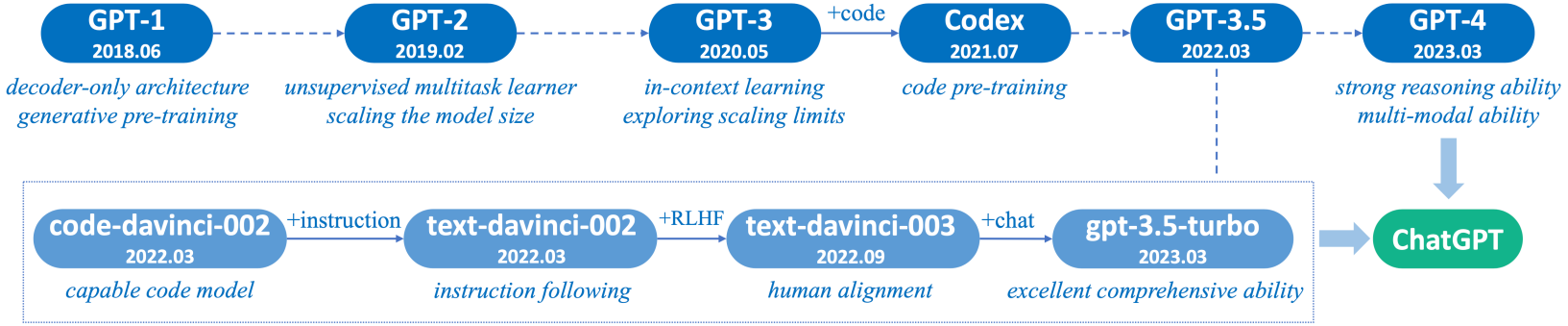



Training objective:

$$l(\theta) = -\sum_{t=1}^{T} \log p_\theta\left(x_t | x_1, \ldots, x_{t-1}\right)$$

https://github.com/huggingface/transformers/tree/main/src/transformers/models/gpt2

# GPT Evolution

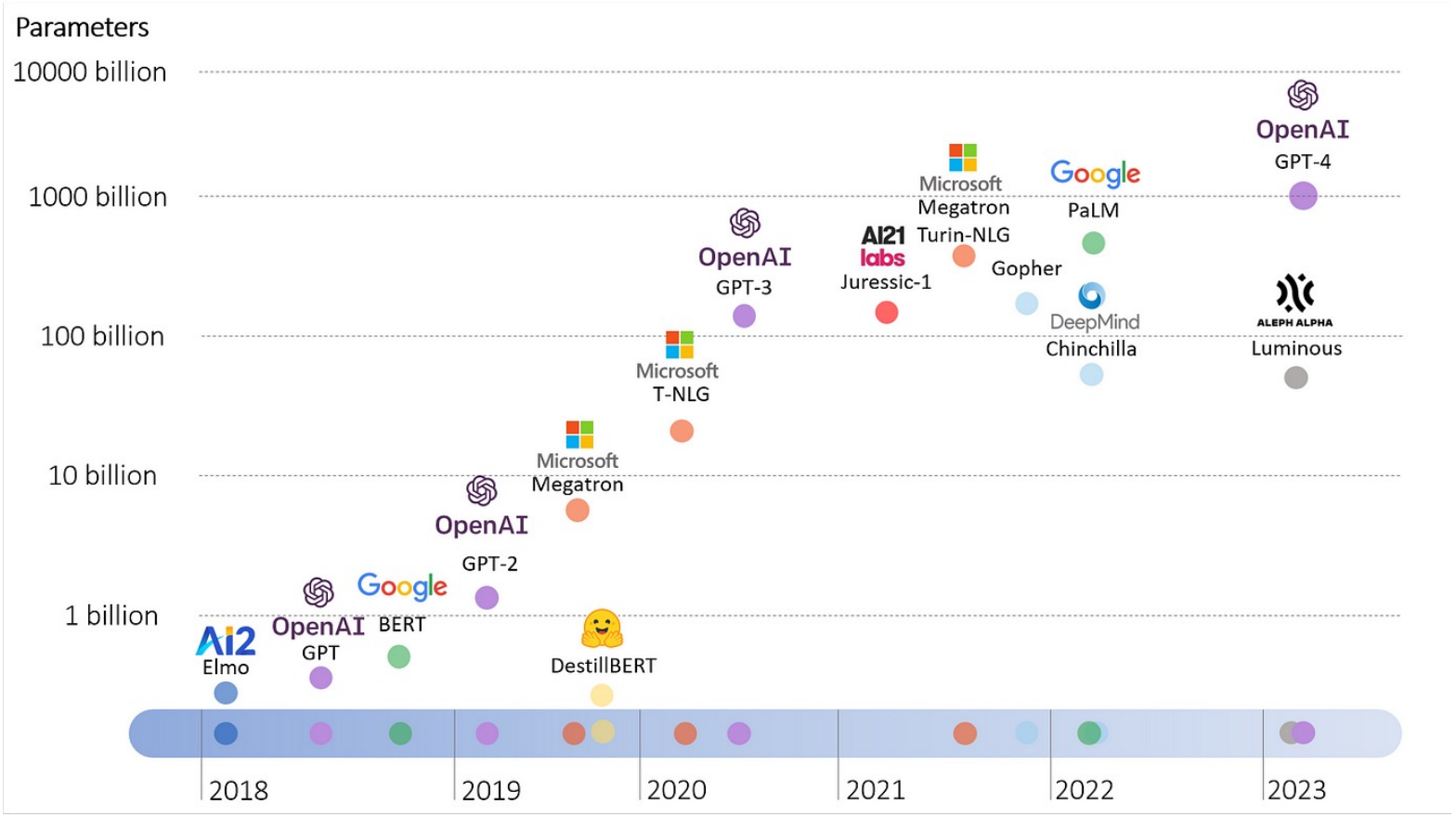



| Simulated exams | GPT-4 estimated percentile | GPT-4 (no vision) estimated percentile | GPT-3.5 estimated percentile |
|---|---|---|---|
| Uniform Bar Exam (MBE+MEE+MPT)[1] | 298 / 400 ~90th | 298 / 400 ~90th | 213 / 400 ~10th |
| LSAT | 163 ~88th | 161 ~83rd | 149 ~40th |
| SAT Evidence-Based Reading & Writing | 710 / 800 ~93rd | 710 / 800 ~93rd | 670 / 800 ~87th |
| SAT Math | 700 / 800 ~89th | 690 / 800 ~89th | 590 / 800 ~70th |
| Graduate Record Examination (GRE) Quantitative | 163 / 170 ~80th | 157 / 170 ~62nd | 147 / 170 ~25th |
| Graduate Record Examination (GRE) Verbal | 169 / 170 ~99th | 165 / 170 ~96th | 154 / 170 ~63rd |
| Graduate Record Examination (GRE) Writing | 4 / 6 ~54th | 4 / 6 ~54th | 4 / 6 ~54th |
| USABO Semifinal Exam 2020 | 87 / 150 99th - 100th | 87 / 150 99th - 100th | 43 / 150 31st - 33rd |
| USNCO Local Section Exam 2022 | 36 / 60 | 38 / 60 | 24 / 60 |
| Medical Knowledge Self-Assessment Program | 75 % | 75 % | 53 % |
| Codeforces Rating | 392 below 5th | 392 below 5th | 260 below 5th |

A Survey of Large Language Models https://arxiv.org/abs/2303.18223

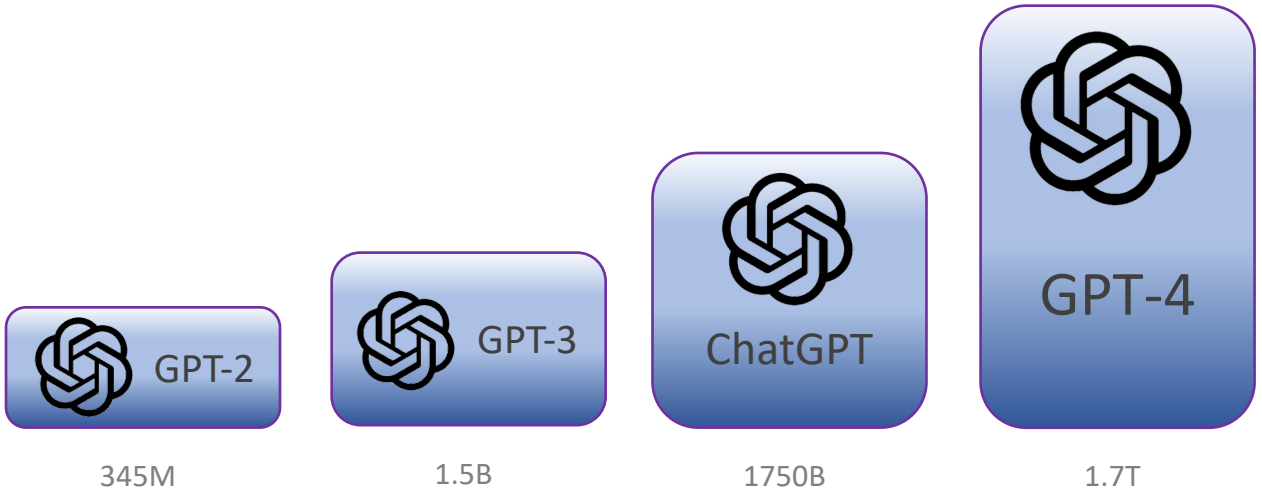
# Make Them Bigger

*Trend of sizes of state-of-the-art NLP models over time on a logarithmic scale*

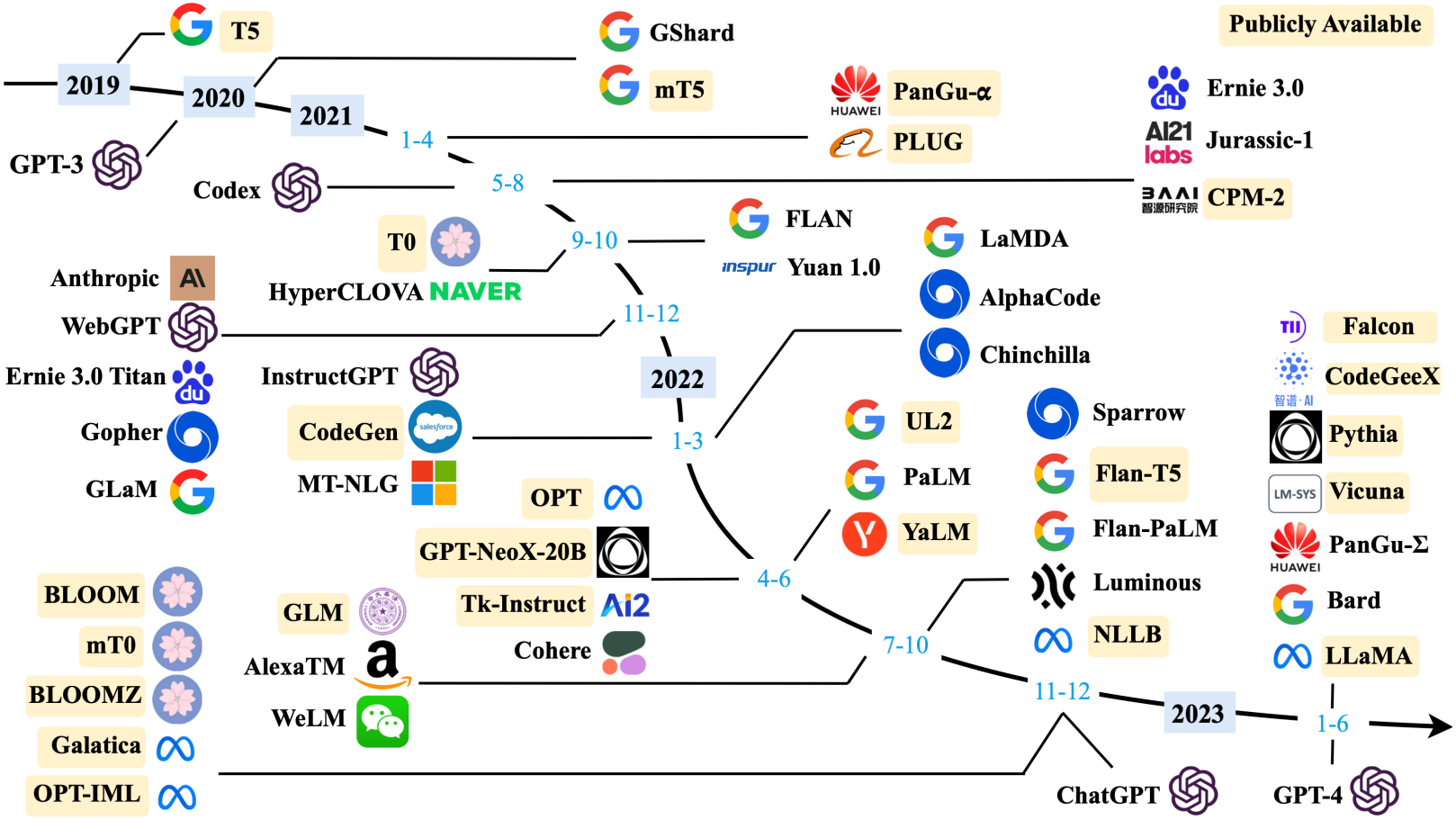


# Large Language Models

Ultra-large pre-trained language models that can be adapted for various tasks

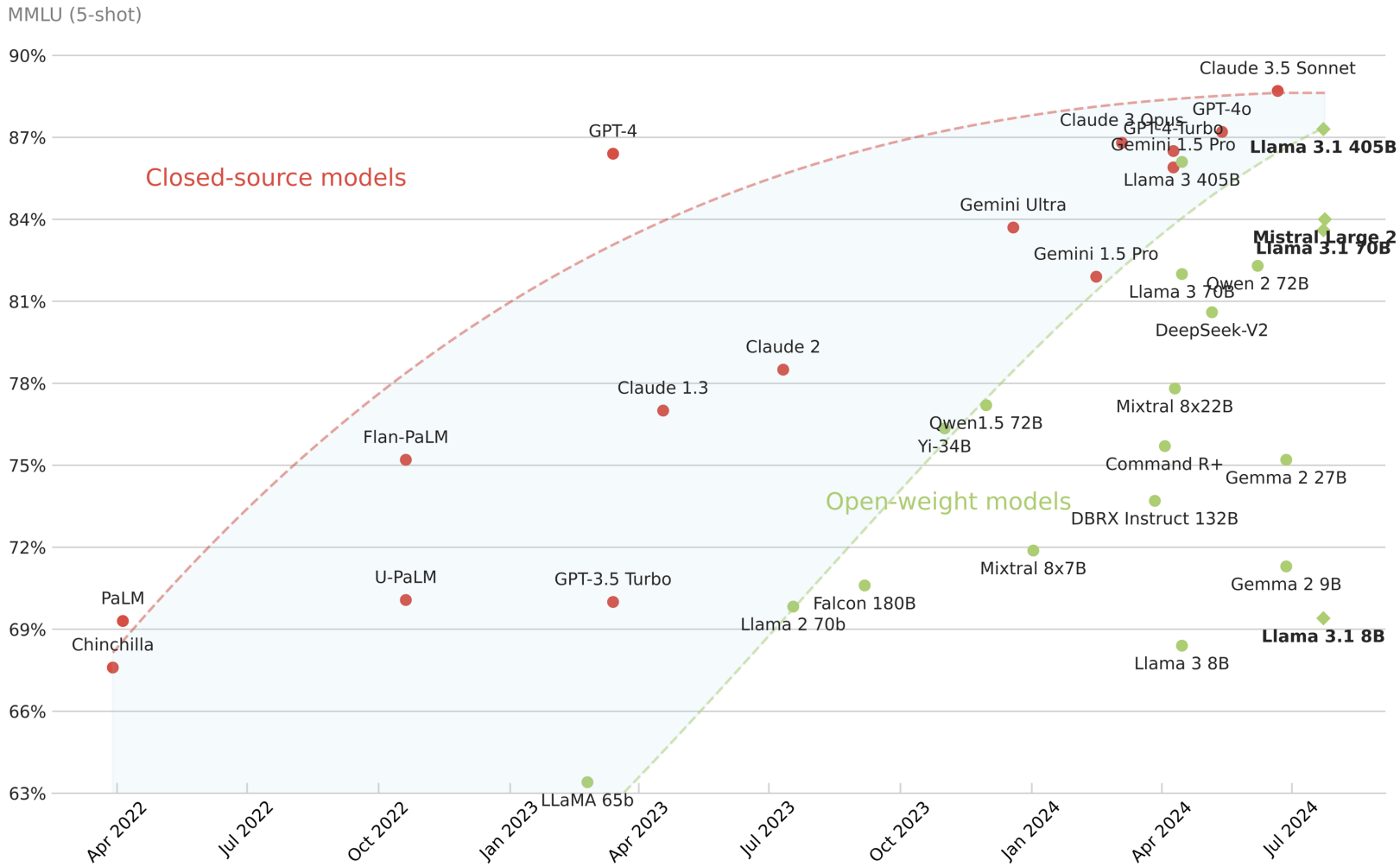


# The LLM Family (2019-)




A Survey of Large Language Models https://arxiv.org/abs/2303.18223

# The LLM Family



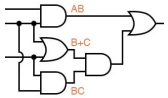
# LLMs: A New Era of Artificial Intelligence

**人工智能(AI)**

研究、开发用于模拟、延伸和扩展人的智能的理论、方法、技术及应用系统的一门新的技术科学



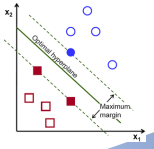
**1950s**
Turning test




**1956**
AI as a new concept

**1970-1980s**
The first winter

**1987-1993**
The second winter




**1990s**
The emergence of machine learning

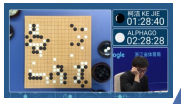


**2006**
The emergence of deep learning

**2012**
AlexNet

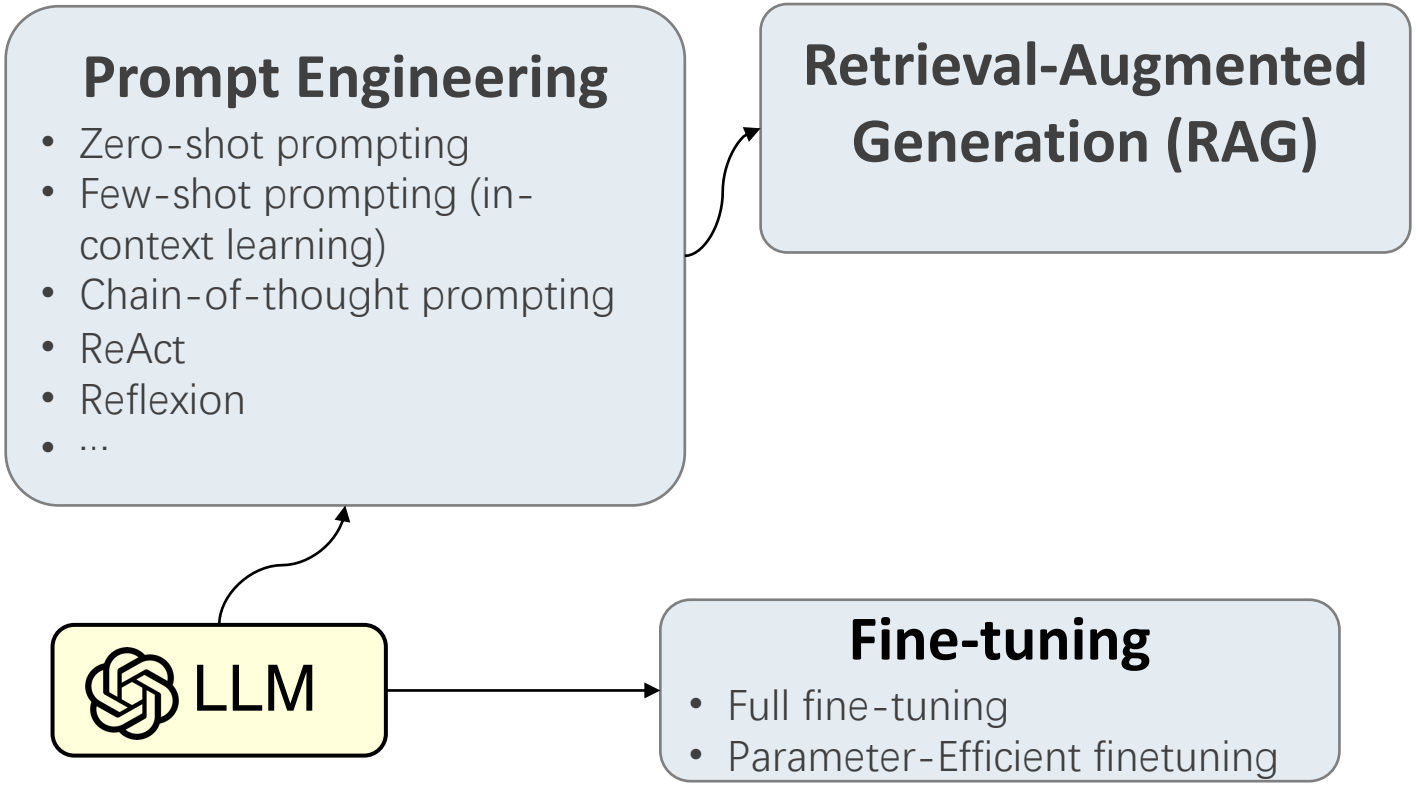



**2016**
AlphaGo

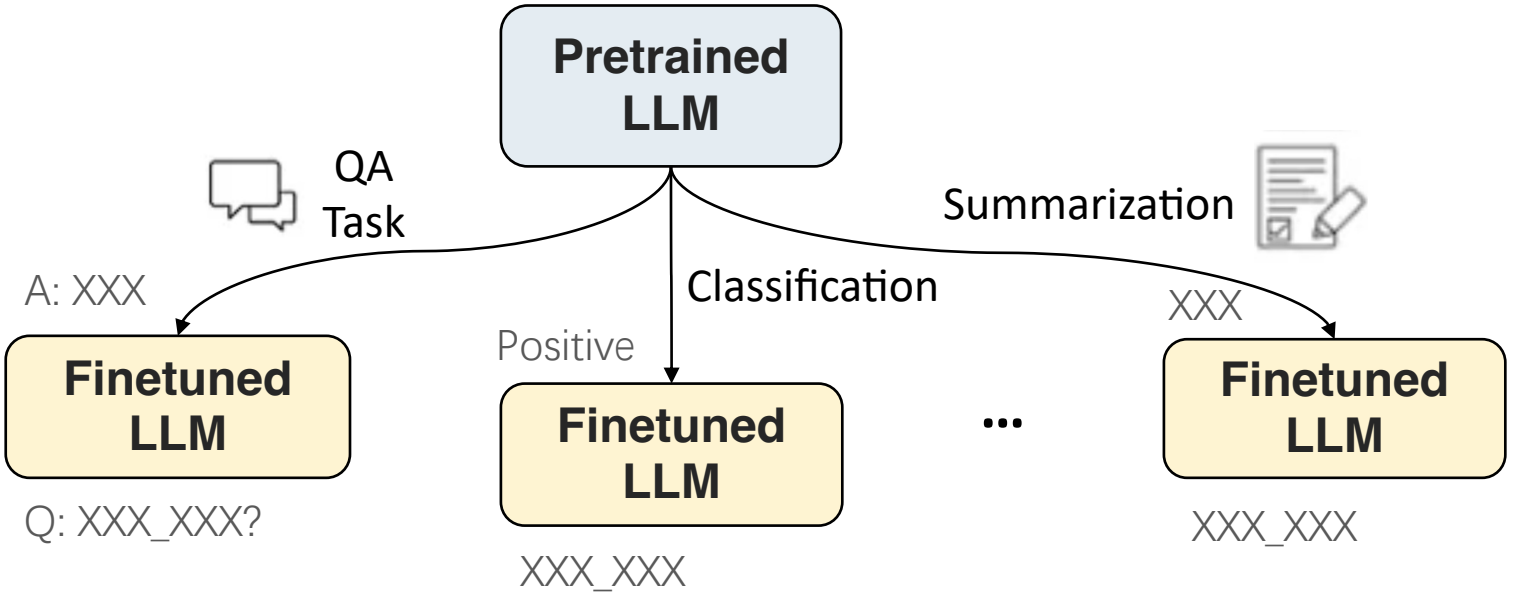


**2022**
ChatGPT

# How Do We Utilize LLMs?



# Fine-tuning

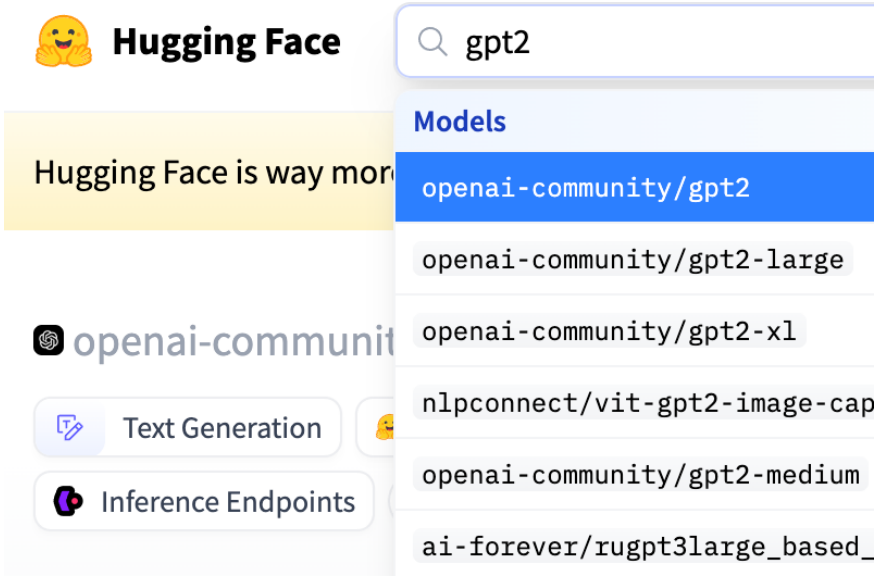


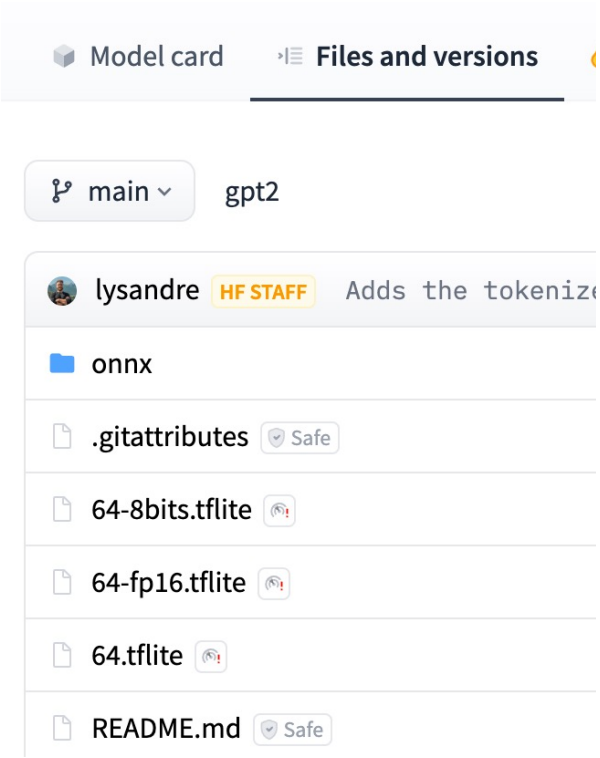

**Problems with full finetuning:**

1. Computationally heavy
2. Overfitting

# Fine-tuning

在Huggingface网站下载模型checkpoint



# Fine-tuning

将模型文件装在指定位置，使用以下方式微调模型

```
from transformers import GPT2LMHeadModel, AutoTokenizer, AutoModel
from transformers import TextDataset, DataCollatorForLanguageModeling
from transformers import Trainer, TrainingArguments

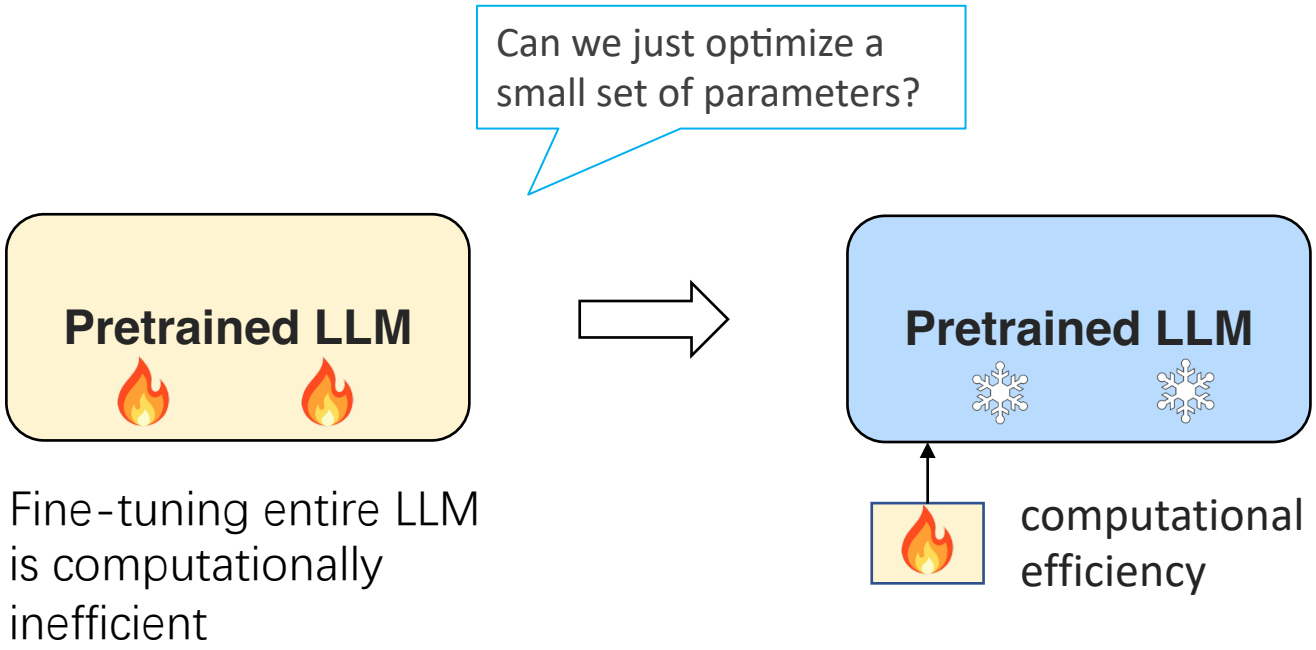
# 指定中文GPT模型和标记器
model_name = "uer/gpt2-large-chinese-cluecorpussmall"
model = GPT2LMHeadModel.from_pretrained('./model_cache/gpt2-chinese-cluecorpussmall/')
tokenizer = AutoTokenizer.from_pretrained('./model_cache/gpt2-chinese-cluecorpussmall/')

# 读取你的文本数据集
dataset_path = "./data/train.txt"

# 使用标记器编码数据集
tokenized_datasets = TextDataset(
    tokenizer=tokenizer,
    file_path=dataset_path,
    block_size=200,  # 适当调整块大小
)

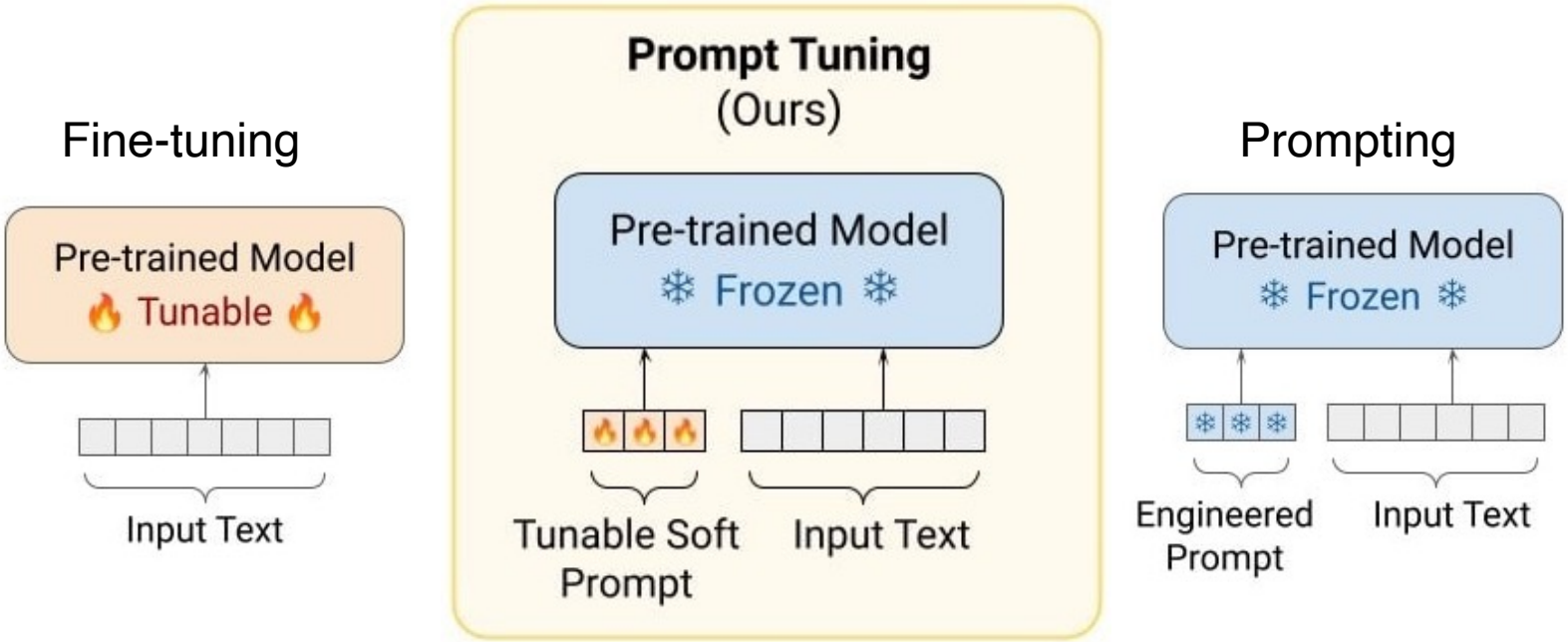
# 准备数据收集器
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False  # 在文本补全中不需要遮蔽语言模型的训练
)
```

# Parameter-Efficient Fine-Tuning (PEFT)



# PEFT – Prompt Tuning

- Prepend the original input with a few tokens (i.e., tunable prompts)
- Freeze pretrained parameters while only optimizing the embeddings of pseudo tokens

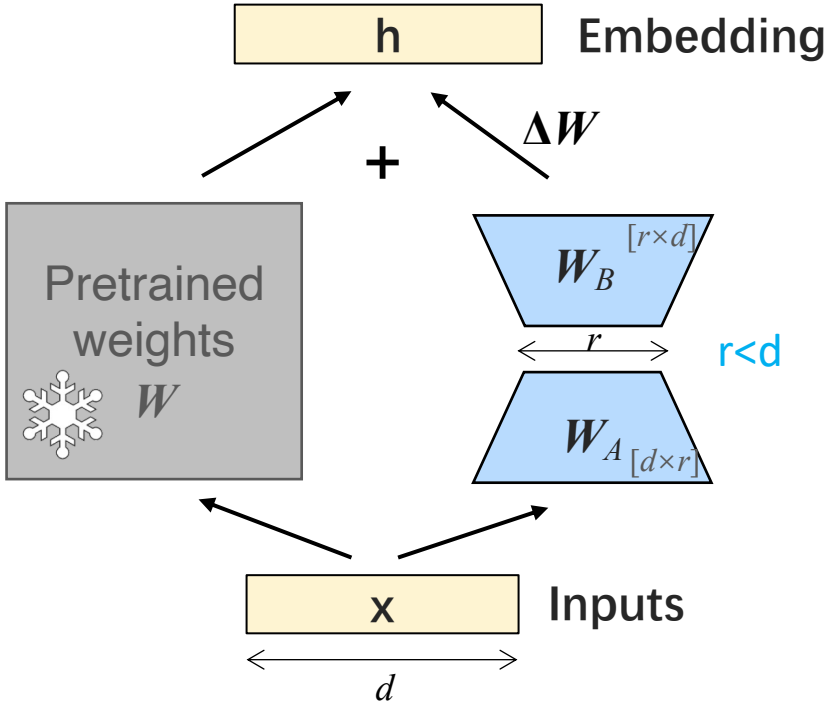



Liu et al. GPT Understands, Too. Arxiv (2021)

# PEFT – LoRA

## Low-Rank Adaptation of Large Language Models

- Represent weight updates using two smaller matrices through low-rank decomposition




LoRA: Low-Rank Adaptation of Large Language Models (2021)

# How to Use PEFT?

① Install PEFT `pip install -U peft`

② Add adapter to the model

```
from transformers import AutoModelForCausalLM
from peft import LoraConfig

model = AutoModelForCausalLM.from_pretrained("google/gemma-2-2b")

lora_config = LoraConfig(
    target_modules=["q_proj", "k_proj"],
    modules_to_save=["lm_head"],
)

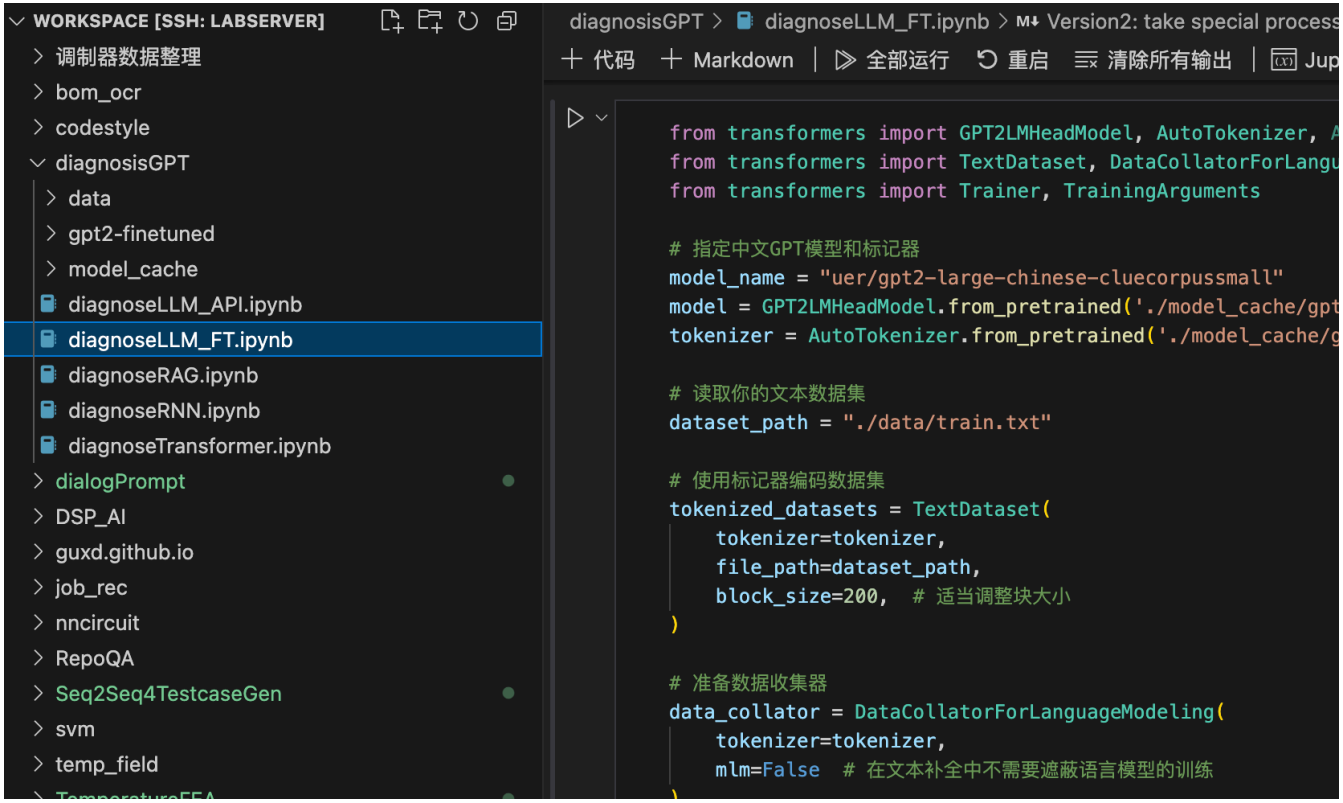
model.add_adapter(lora_config)
trainer = Trainer(model=model, ...)
trainer.train()
```

https://huggingface.co/docs/transformers/peft

# TIME for Coding

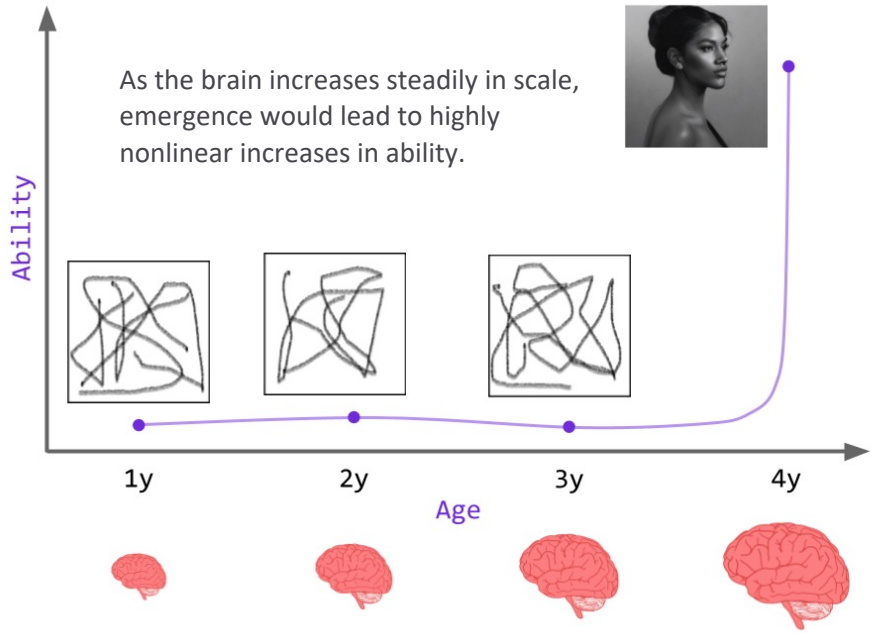


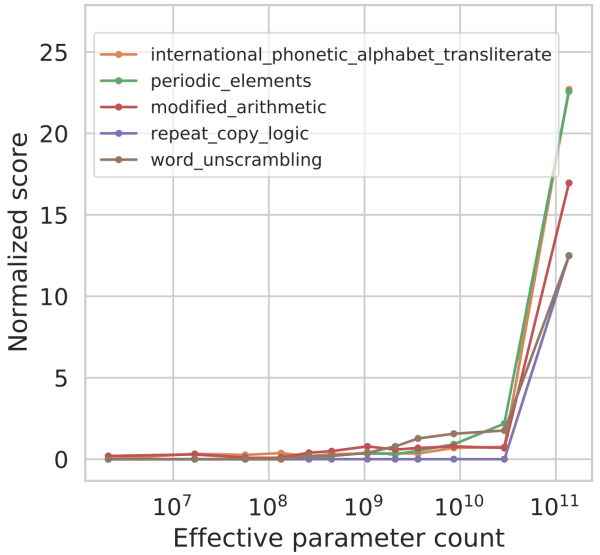

**Tutorial**: AI Diagnosis By Fine-tuning LLMs



# Emergent (涌现) Ability of LLMs

**The sudden appearance of novel behaviors (beyond text completion) as the size of LLM grows**

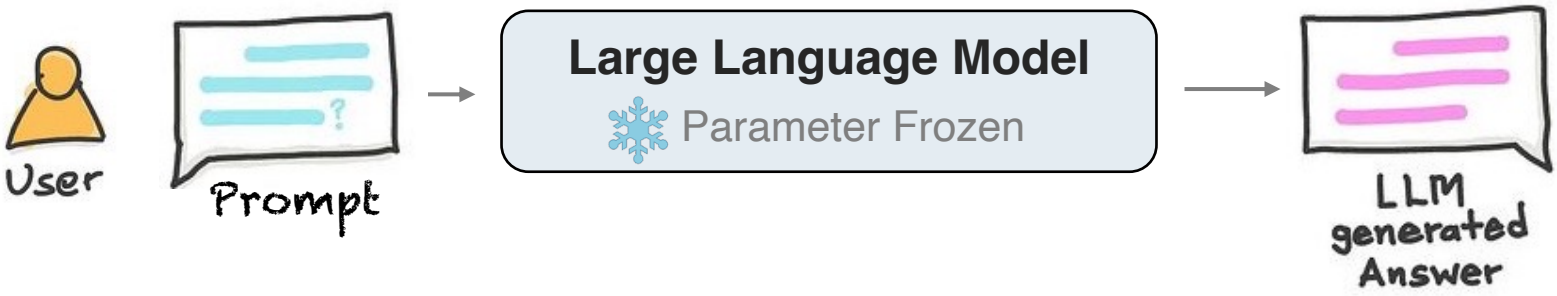



Emergent ability allows LLMs to perform tasks **without task-specific training**, *just by phrasing them in terms of natural language*.

# Prompting

- Prompting LLMs with a task description.




**Simple Prompt**

Q: How many colors are in the rainbow?

A: There are seven colors in the rainbow.

**Instruction Prompt**

Compose a motivational poem to make me happy:

Life is full of ups and downs,
But don't let it get you down.
You can't always get what you want,
But if you try sometimes, you might find,
You get what you need.
If you want to make a difference,
Start by doing what's right.

**Complicated Prompt**

| Company Name | Number of Employees | Year Established | IPO Date | Share Price |
| Providence Inc. | 250 | 1990 | 25th August 1990 | $0.90 |
| Grant Corporation | 2000 | 1890 | 21st September 1920 | $115.90 |
| Rusty Metalworks | 12459 | 1946 | 12th September 1986 | $15.23 |

Q: Which company has the most employees?
A: Rusty Metalworks
Q: What is the share price and IPO date of Grant Corporation?
A: $115.90 and 21st September 1920

# Prompting



- Prompting LLMs with a task description.

```
from transformers import pipeline
import torch

pipeline = pipeline(task="text-generation", model="mistralai/Mistal-7B-Instruct-v0.1", torch_dtype=t
prompt = """Classify the text into neutral, negative or positive.
Text: This movie is definitely one of my favorite movies of its kind. The interaction between respec
Sentiment:
"""

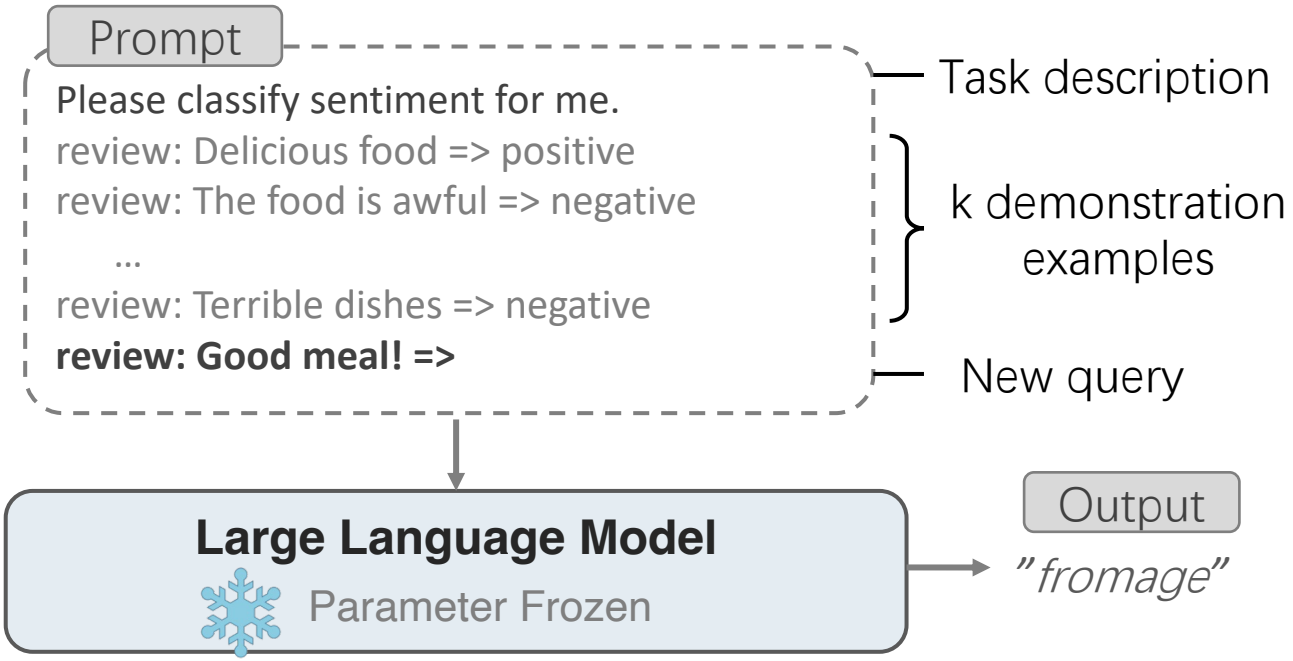
outputs = pipeline(prompt, max_new_tokens=10)
for output in outputs:
    print(f"Result: {output['generated_text']}")
Result: Classify the text into neutral, negative or positive.
Text: This movie is definitely one of my favorite movies of its kind. The interaction between respec
Sentiment:
Positive
```

https://huggingface.co/docs/transformers/tasks/prompting

# Few-Shot Prompting

- Prompting LLMs with a few demonstration examples

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.



# Few-shot Prompting

- **In-Context Learning**: Learning by mimicking demonstration examples in the input context.
- No parameter update






```
Input: 2014-06-01
Output: !06!01!2014!
Input: 2007-12-13
Output: !12!13!2007!
Input: 2010-09-23
Output: !09!23!2010!
Input: 2005-07-23
Output: !07!23!2005!
```

*in-context examples*

*test example*

*model completion*

# Few-Shot Prompting

## Practice

```
from transformers import pipeline
import torch

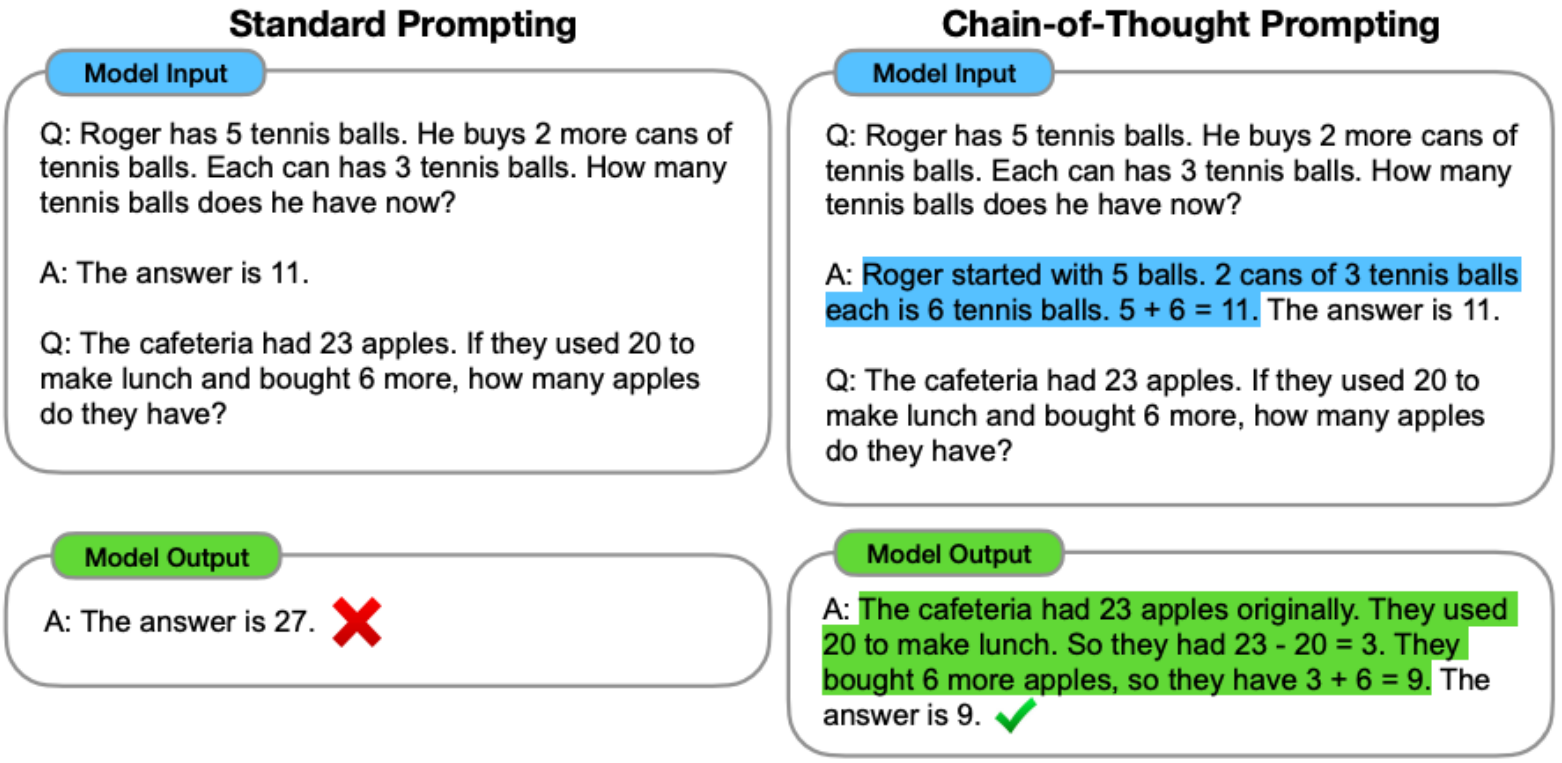
pipeline = pipeline(model="mistralai/Mistral-7B-Instruct-v0.1", torch_dtype=torch.bfloat16, device_m
prompt = """Text: The first human went into space and orbited the Earth on April 12, 1961.
Date: 04/12/1961
Text: The first-ever televised presidential debate in the United States took place on September 28,
Date:"""

outputs = pipeline(prompt, max_new_tokens=12, do_sample=True, top_k=10)
for output in outputs:
    print(f"Result: {output['generated_text']}")
Result: Text: The first human went into space and orbited the Earth on April 12, 1961.
Date: 04/12/1961
Text: The first-ever televised presidential debate in the United States took place on September 28,
Date: 09/28/1960
```

https://huggingface.co/docs/transformers/tasks/prompting

# Chain-of-Thought Prompting

- Prompting with step-by-step reasoning that lead to the final answer for a problem.




Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. (2022)

# Chain-of-Thought Prompting

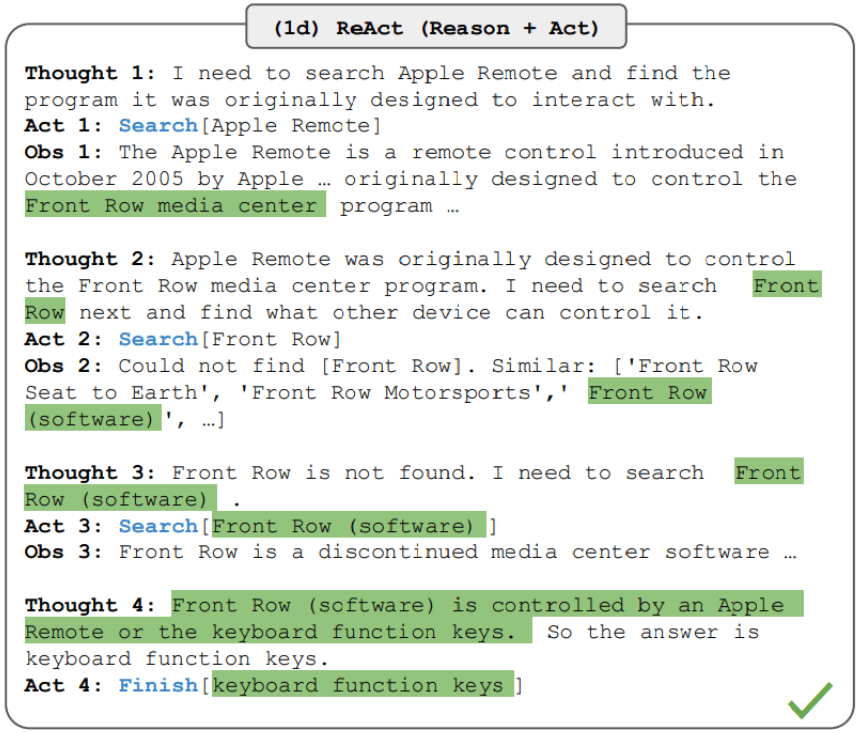
```
from transformers import pipeline
import torch

pipeline = pipeline(model="mistralai/Mistral-7B-Instruct-v0.1", torch_dtype=torch.bfloat16, device_m
prompt = """Let's go through this step-by-step:
1. You start with 15 muffins.
2. You eat 2 muffins, leaving you with 13 muffins.
3. You give 5 muffins to your neighbor, leaving you with 8 muffins.
4. Your partner buys 6 more muffins, bringing the total number of muffins to 14.
5. Your partner eats 2 muffins, leaving you with 12 muffins.
If you eat 6 muffins, how many are left?"""

outputs = pipeline(prompt, max_new_tokens=20, do_sample=True, top_k=10)
for output in outputs:
    print(f"Result: {output['generated_text']}")
Result: Let's go through this step-by-step:
1. You start with 15 muffins.
2. You eat 2 muffins, leaving you with 13 muffins.
3. You give 5 muffins to your neighbor, leaving you with 8 muffins.
4. Your partner buys 6 more muffins, bringing the total number of muffins to 14.
5. Your partner eats 2 muffins, leaving you with 12 muffins.
If you eat 6 muffins, how many are left?
Answer: 6
```

# ReAct

LLMs are used to generate both *reasoning traces* and *task-specific actions* in an interleaved manner.



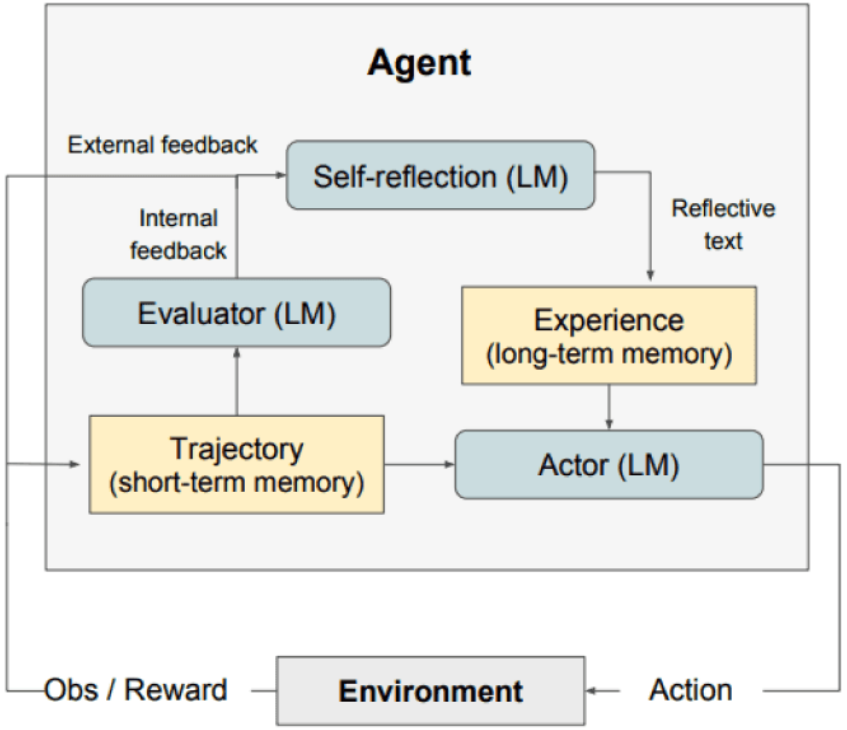

The ReAct framework can allow LLMs to interact with external tools to retrieve additional information that leads to more reliable and factual responses.

https://arxiv.org/abs/2210.03629

# Reflexion

Reflexion is a framework to reinforce language-based agents through linguistic feedback.

‘verbal‘ reinforcement learning by LLMs




https://arxiv.org/pdf/2303.11366.pdf

# 使用LLM API进行大模型编程

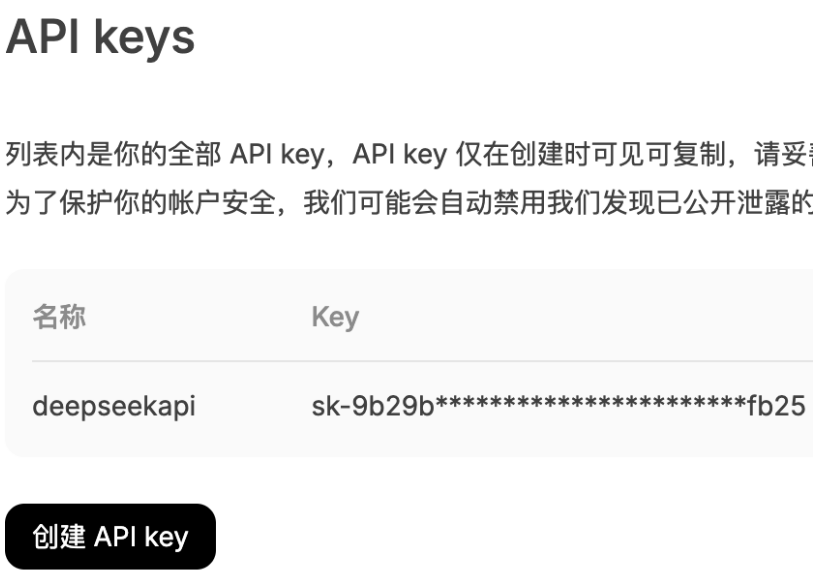
在大模型官网通常有API平台链接



# 使用LLM API进行大模型编程

## 充值，获得API Key



# 使用LLM API进行大模型编程

## 在程序中使用API key直接调用大模型

```python
import json
import requests
# DeepSeek API 端点
DEEPSEEK_API_URL = "https://api.deepseek.com/v1/chat/completions"
# API 密钥
DEEPSEEK_API_KEY = "sk-9b**********25"

# 调用 DeepSeek API 生成文本
def generate_text(prompt, model="deepseek-chat", max_tokens=200):
    headers = {
        "Authorization": f"Bearer {DEEPSEEK_API_KEY}",
        "Content-Type": "application/json"
    }
    data = {
        "model": model,
        "messages": [
            {"role": "system", "content": "你是一个医疗助手，帮助用户生成医疗病例文本。"},
            {"role": "user", "content": prompt}
        ],
        "max_tokens": max_tokens,
        "temperature": 0.7
    }
    response = requests.post(DEEPSEEK_API_URL, headers=headers, json=data)
    # 处理响应
    if response.status_code == 200:
        result = response.json()['choices'][0]['message']['content']
    else:
        result = f"请求失败，状态码：{response.status_code}, {response.text}"
    return result
```

```python
# 示例：生成医疗病例文本
prompt = "患者主诉头痛、发热，请生成一份详细的医疗病例。"
generated_text = generate_text(prompt)
print("生成的医疗病例文本:\n", generated_text)
```

```
生成的医疗病例文本:
 **医疗病例**

**患者信息：**
- 姓名：[患者姓名]
- 性别：[患者性别]
- 年龄：[患者年龄]
- 职业：[患者职业]
- 联系方式：[患者联系方式]
- 就诊日期：[就诊日期]

**主诉：**
患者主诉头痛、发热。

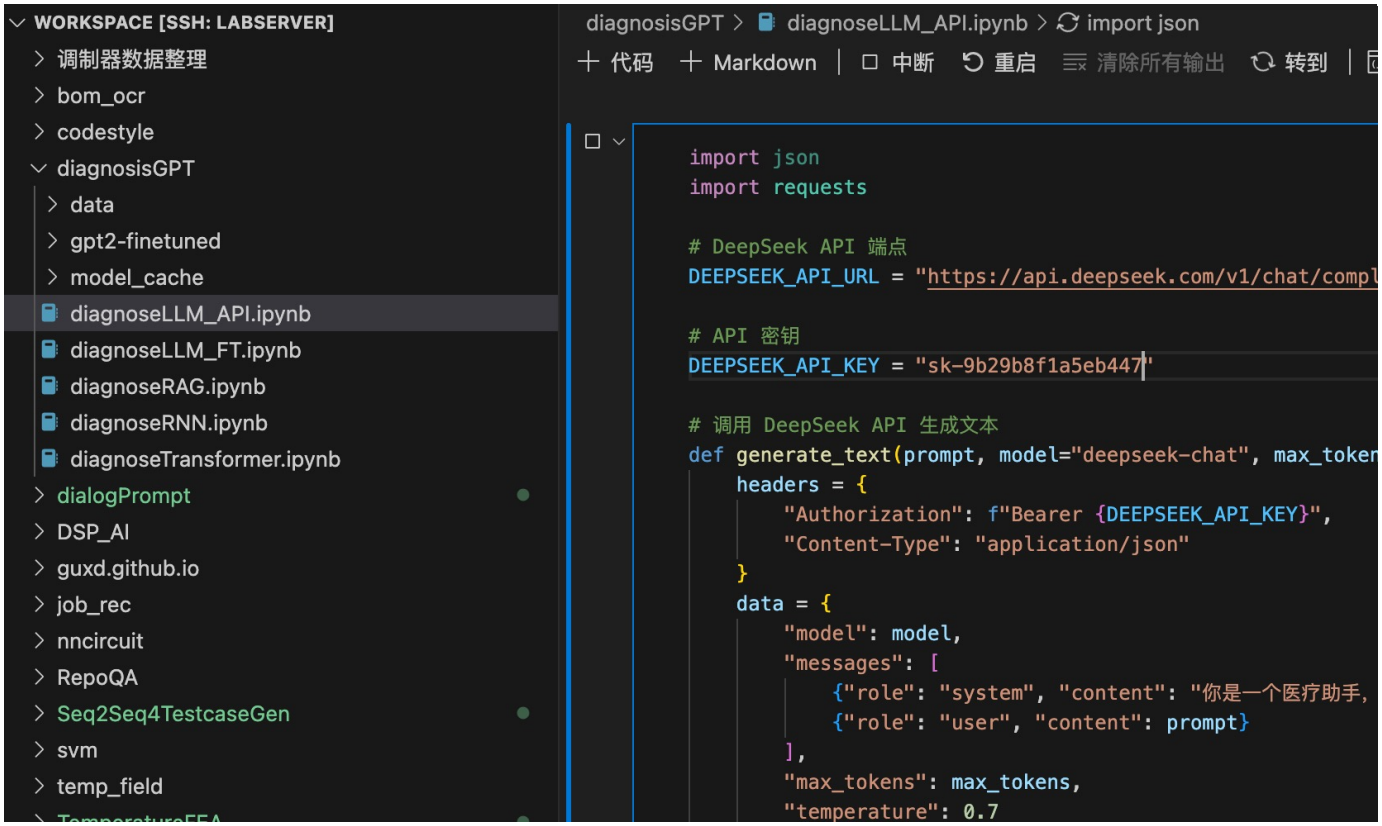
**现病史：**
患者于[发病时间]开始出现头痛，疼痛性质为[描述疼痛性质，如钝痛、刺

**既往史：**
患者既往有[描述既往
```

# TIME for Coding



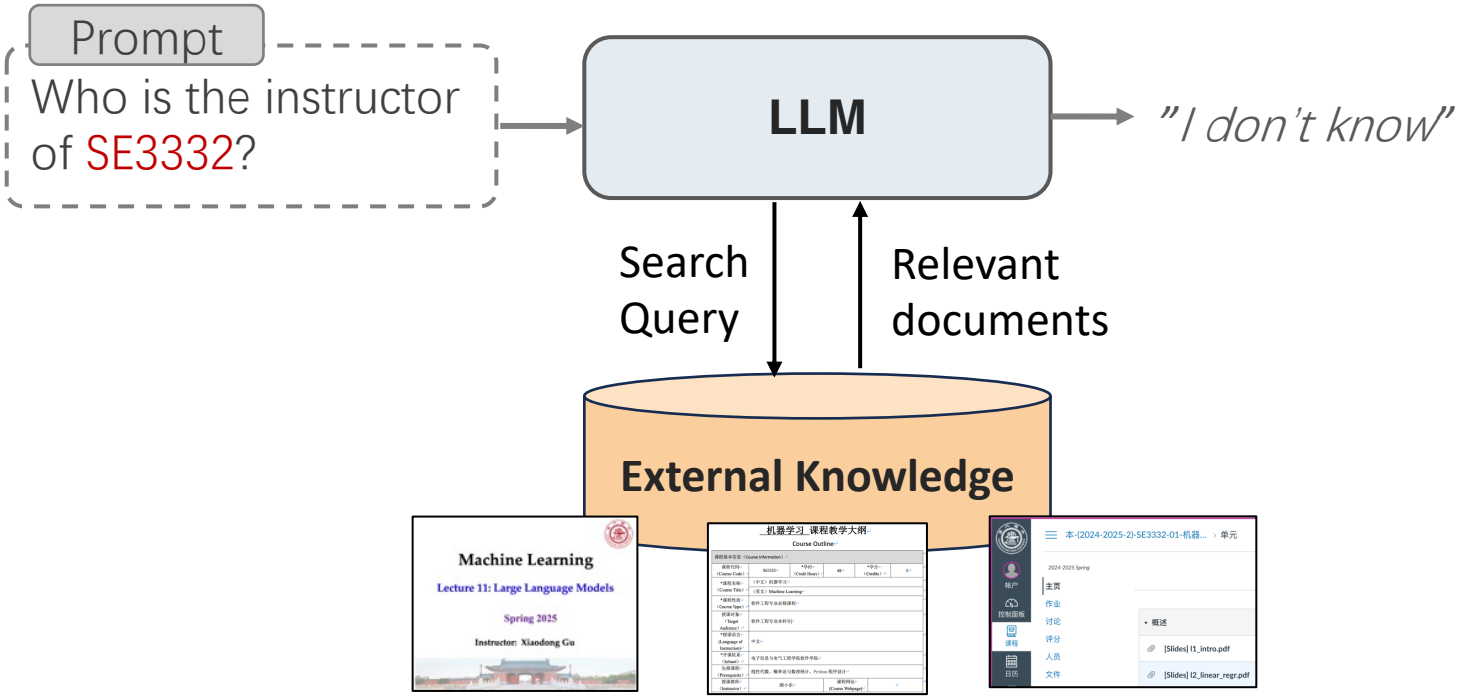

**Tutorial**: AI Diagnosis Using LLM API



# Retrieval-Augmented Generation (RAG)

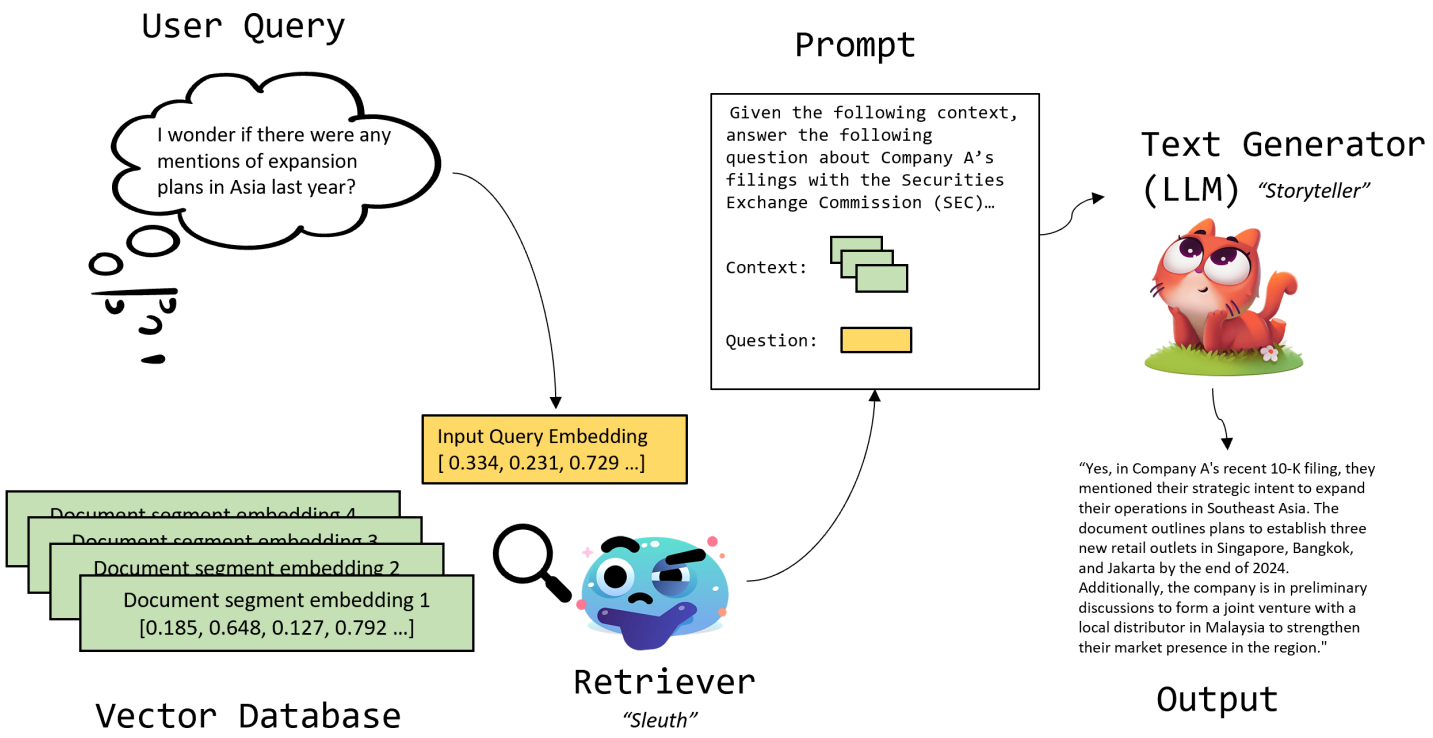
## Why RAG?

LLMs may lack access to the most up-to-date or domain-specific knowledge



# RAG – Overview

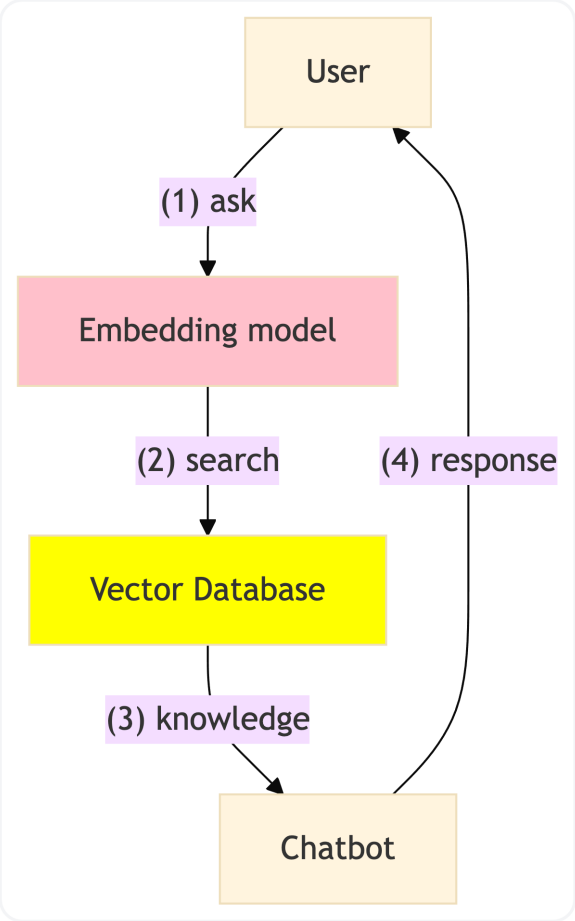
## Augmenting LLM knowledge through retrieving external documents



# RAG – Framework

A RAG system consists of three key components:

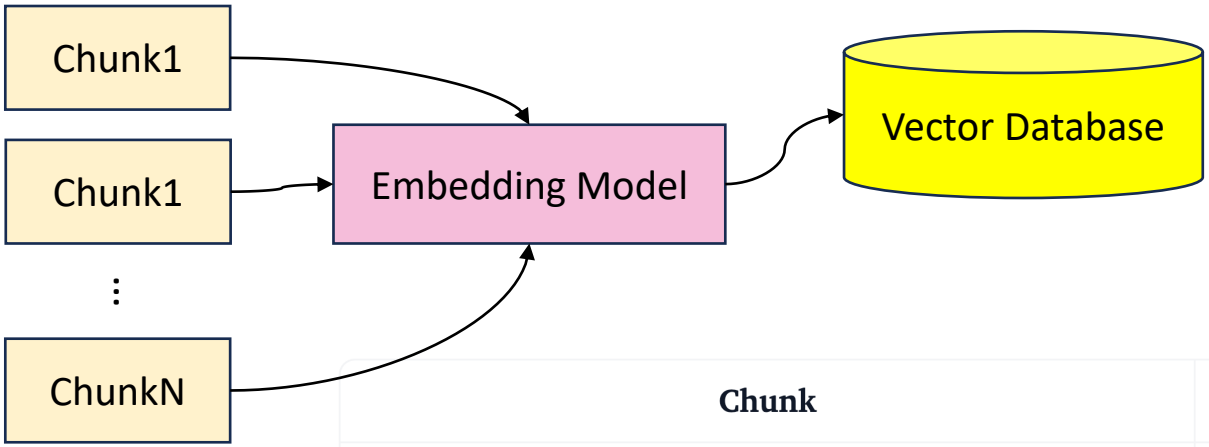
1. **Embedding model**: converts input text into embeddings for quick retrieval.
2. **Vector database**: stores knowledge and its corresponding embedding vectors.
3. **LLM**: generates responses based on retrieved knowledge.




https://huggingface.co/blog/ngxson/make-your-own-rag

# RAG – Pipeline

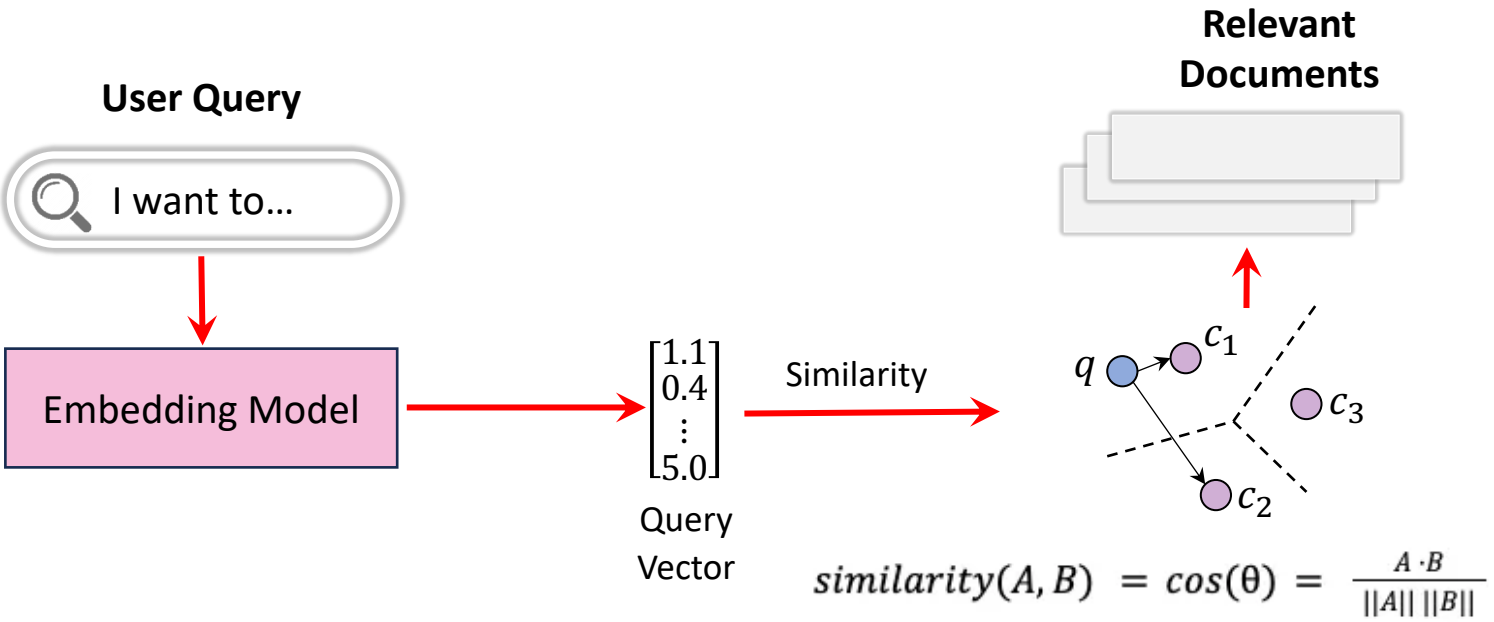
**Index Phase:** breaking documents into small **chunks** and embedding them into vectors that can be efficiently searched during generation.




| Chunk | Embedding Vector |
|---|---|
| Italy and France produce over 40% of all wine in the world. | \[0.1, 0.04, -0.34, 0.21, ...\] |
| The Taj Mahal in India is made entirely out of marble. | \[-0.12, 0.03, 0.9, -0.1, ...\] |
| 90% of the world's fresh water is in Antarctica. | \[-0.02, 0.6, -0.54, 0.03, ...\] |
| ... | ... |

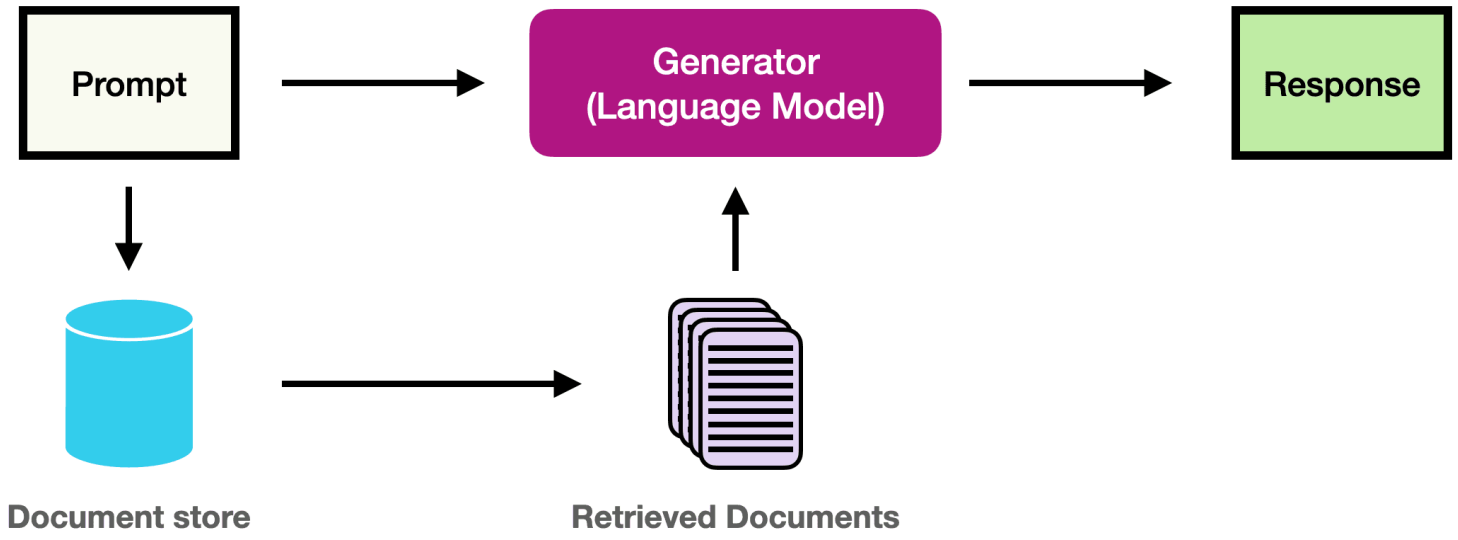
# RAG – Pipeline

**Retrieval Phase:** For a new query, calculate the query vector, and compare it against the vectors in the database to find the top-K relevant chunks.



# RAG – Pipeline

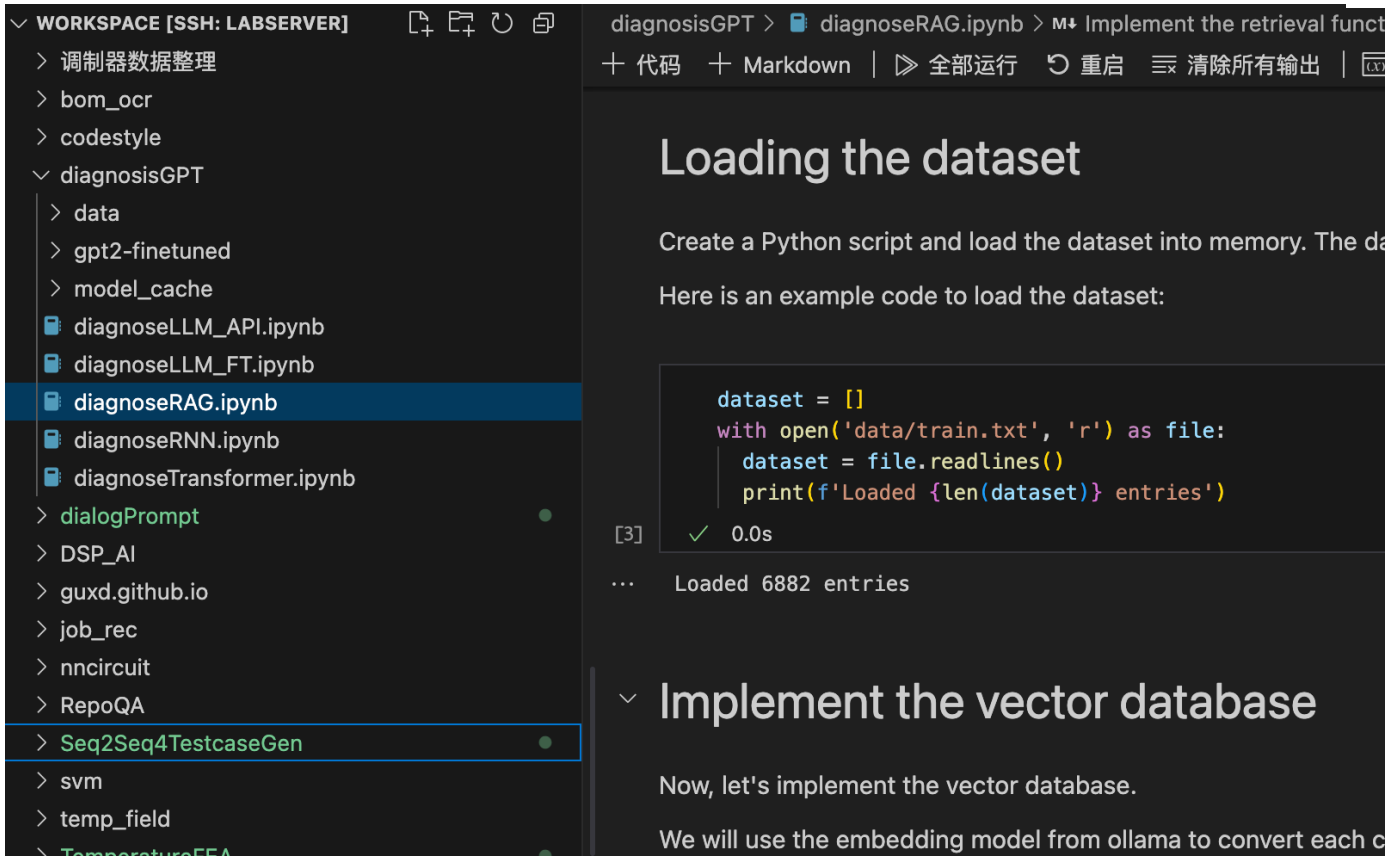
**Generation Phase:** Finally, the retrieved chunks will be augmented to the original prompt to the LLM to generate a response.



# TIME for Coding




**Tutorial**: AI Diagnosis Using RAG



# 常用的LLM工具




- Ollama — 本地化部署大模型
- AnythingLLM—构建LLM本地知识管理引擎
- LlamaFactory 一站式大模型微调平台 (https://github.com/hiyouga/LLaMA-Factory)
- LangChain开发由LLM支持的应用程序的框架 (https://github.com/langchain-ai/langchain)

# Ollama

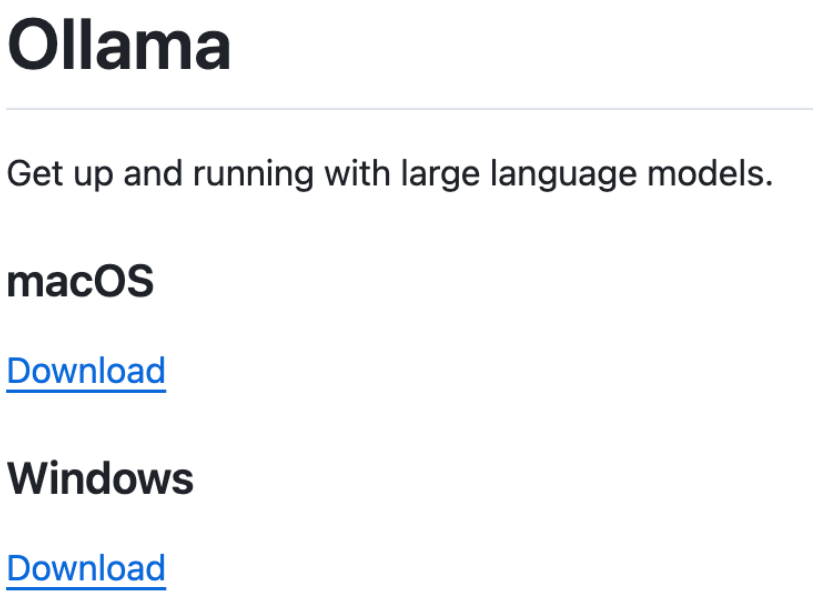
Ollama 用于在本地运行大语言模型。

Github主页：https://github.com/ollama/ollama



# Ollama

下载：直接在Github主页下载对应安装包



# Ollama

## 运行：采用命令行运行

```
(base) xiaodong-iMacdeiMac:~ xiaodong$ ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve       Start ollama
  create      Create a model from a Modelfile
  show        Show information for a model
  run         Run a model
  stop        Stop a running model
  pull        Pull a model from a registry
  push        Push a model to a registry
  list        List models
  ps          List running models
  cp          Copy a model
  rm          Remove a model
  help        Help about any command
```

**快速运行：ollama run llama3.2:1b**

```
(base) xiaodong-iMacdeiMac:~ xiaodong$ ollama run llama3.2:1b
>>> why is the sky blue
The color of the sky is a fascinating topic, and there are seve
blue to our eyes. Here's a simplified explanation:

When sunlight enters Earth's atmosphere, it encounters tiny mol
nitrogen (N2) and oxygen (O2). These gas molecules scatter the
they scatter shorter (blue) wavelengths more than longer (red)

This is known as Rayleigh scattering, named after the British p
first described it in the late 19th century. As a result of thi
is distributed throughout the atmosphere, giving the sky its bl

There are a few other factors that can contribute to the appare
different times and locations:

1. **Dust and water vapor**: Tiny particles in the air, such as
vapor, can scatter light in a way that enhances the blue color.
```

# AnythingLLM

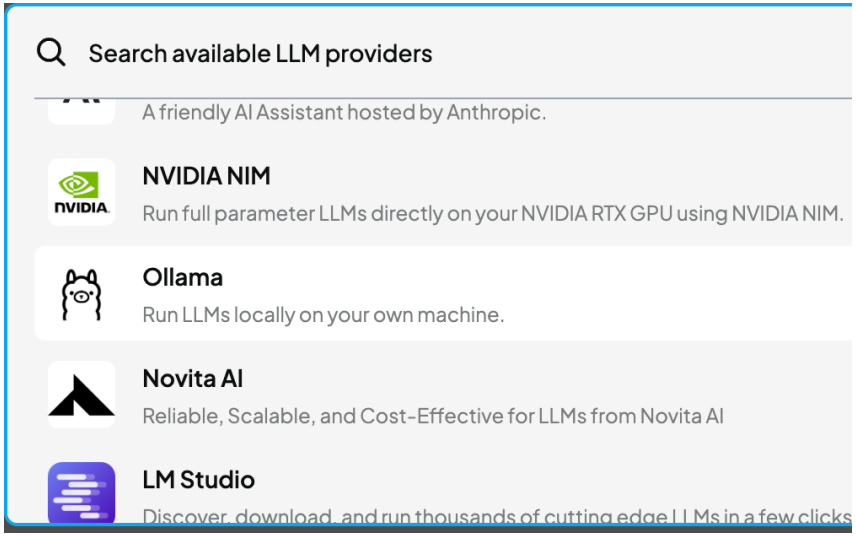
AnythingLLM 是一个文档问答框架，支持将本地文档与 LLM 结合。

**快速安装**：直接在官网下载客户端
https://anythingllm.com/desktop



# AnythingLLM

**设置大模型**：在工作区设置，代理配置，选择ollama, 模型选择ollma中已经下载的模型，如llama3.2:1b

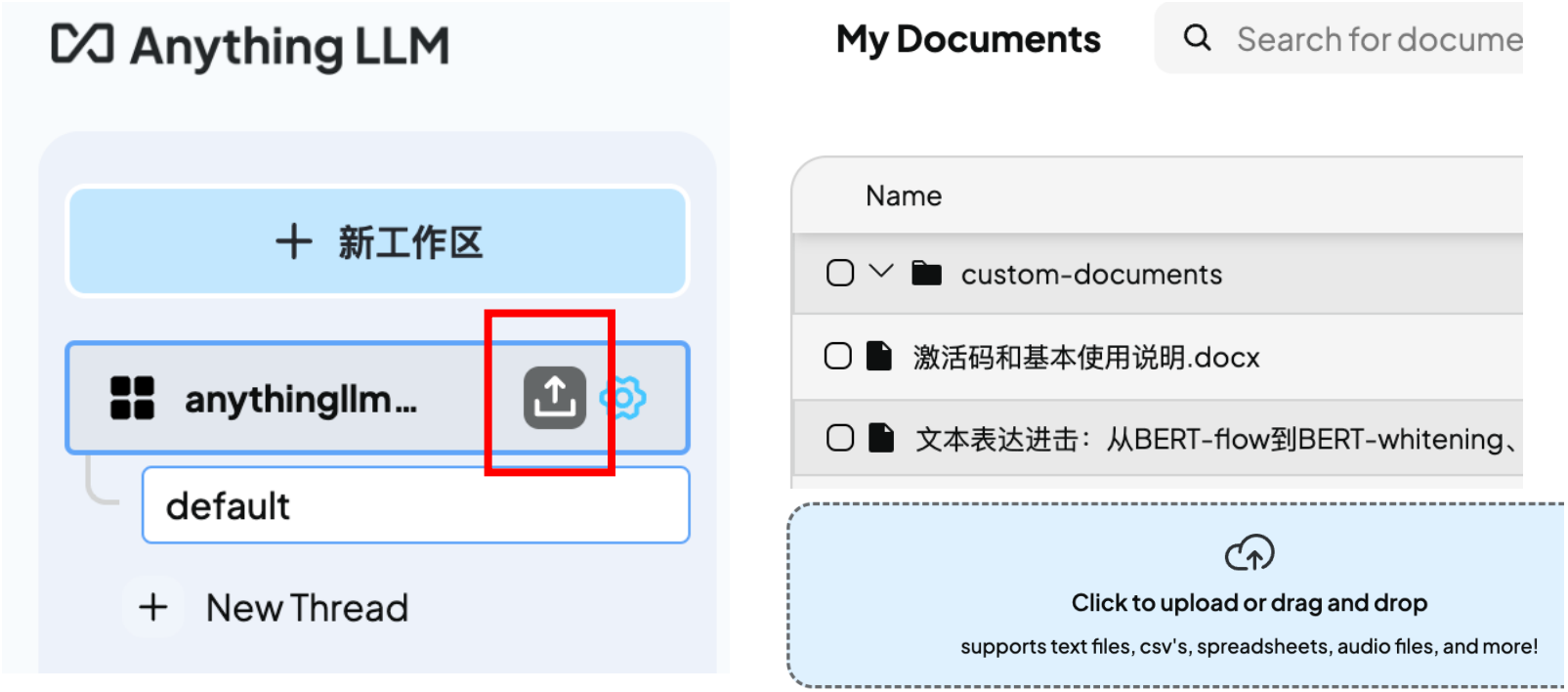





# AnythingLLM

## 设置本地向量数据库

在工作区点击“上传”样式的图标，进入本地文件上传页面，点击上传图标上传文件，并移动到workspace, 构建embedding



# AnythingLLM

## 开始大模型交互

what is BERT-flow

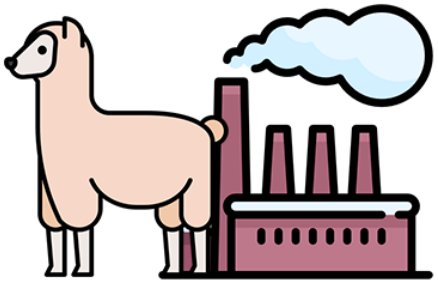
BERT-flow 是一个用于在BERT 中生成向量空间的变换模型。它通过将BERT 的

BERT-flow 的基本思想是利用 BERT 的预测结果作为输入并将其使用为流变换

BERT-flow 的好处在于，它可以通过使用 BERT 流变换来生成可逆的映射变换

BERT-flow 是一个用于在BERT 中生成向量空间的变换模型，其结构如下：

# LLaMA-Factory



## LLaMA-Factory

### Easy and Efficient LLM Fine-Tuning

**Supported Models**

| Model | Model size | Template |
|---|---|---|
| Baichuan 2 | 7B/13B | baichuan2 |
| BLOOM/BLOOMZ | 560M/1.1B/1.7B/3B/7.1B/176B | - |
| ChatGLM3 | 6B | chatglm3 |
| Command R | 35B/104B | cohere |
| DeepSeek (Code/MoE) | 7B/16B/67B/236B | deepseek |
| Falcon | 7B/11B/40B/180B | falcon |
| Gemma/Gemma 2/CodeGemma | 2B/7B/9B/27B | gemma |
| GLM-4 | 9B | glm4 |
| Index | 1.9B | index |
| InternLM2/InternLM2.5 | 7B/20B | intern2 |
| Llama | 7B/13B/33B/65B | - |

**Supported Training Approaches**

| Approach | Full-tuning | Freeze-tuning | LoRA | QLoRA |
|---|---|---|---|---|
| Pre-Training | ✅ | ✅ | ✅ | ✅ |
| Supervised Fine-Tuning | ✅ | ✅ | ✅ | ✅ |
| Reward Modeling | ✅ | ✅ | ✅ | ✅ |
| PPO Training | ✅ | ✅ | ✅ | ✅ |
| DPO Training | ✅ | ✅ | ✅ | ✅ |
| KTO Training | ✅ | ✅ | ✅ | ✅ |
| ORPO Training | ✅ | ✅ | ✅ | ✅ |
| SimPO Training | ✅ | ✅ | ✅ | ✅ |

# 使用LLaMA-Factory进行微调



## 1. 前置准备

安装LLaMAFactory

```
git clone https://github.com/hiyouga/LLaMA-Factory.git
conda create -n llama_factory python=3.10
conda activate llama_factory
cd LLaMA-Factory
pip install -e '.[torch,metrics]'
```

下载模型

以Meta-Llama-3-8B-Instruct为例，通过huggingface 下载（可能需要先提交申请通过）

```
git clone https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
```

modelscope 下载（适合中国大陆网络环境）

```
git clone https://www.modelscope.cn/LLM-Research/Meta-Llama-3-8B-Instruct.git
```

教程见：https://zhuanlan.zhihu.com/p/695287607

# 使用LLaMA-Factory进行微调

## 2. 准备微调数据

dataset_info.json 包含了所有可用的数据集。如果您希望使用自定义数据集，请**务必**在 `dataset_info.json` 文件中添加*数据集描述*，并通过修改 `dataset: 数据集名称` 配置来使用数据集。

```
{
  "identity": {
    "file_name": "identity.json"
  },
  "alpaca_en_demo": {
    "file_name": "alpaca_en_demo.json"
  },
  "alpaca_zh_demo": {
    "file_name": "alpaca_zh_demo.json"
  },
```

./data/dataset_info.json

./data/alpaca_zh_demo.json

```
[
  {
    "instruction": "识别并解释给定列表中的两个科学理论：细胞理论和日心说。",
    "input": "",
    "output": "细胞理论是生物科学的一个理论，它认为所有生命体都是由微小的基本单元—细胞所构成。
  },
  {
    "instruction": "输入三支篮球队的名称并生成一个适当的口号。",
    "input": "输入：俄克拉荷马城雷霆队，芝加哥公牛队，布鲁克林网队。",
    "output": "输出： \"雷霆，公牛和网队：各显神通，角逐群雄！\""
  },
  {
    "instruction": "比较以下两位著名人物，使用特征列表并以句子形式提供输出。",
    "input": "人物1：阿尔伯特·爱因斯坦\n人物2：斯蒂芬·霍金\n特征：智力、教育、对世界的影响",
    "output": "阿尔伯特·爱因斯坦和斯蒂芬·霍金都是拥有极其出色智力的人物。两人都取得过非常高的教
  },
```

# 使用LLaMA-Factory进行微调

## 1. 启动微调

命令行方式

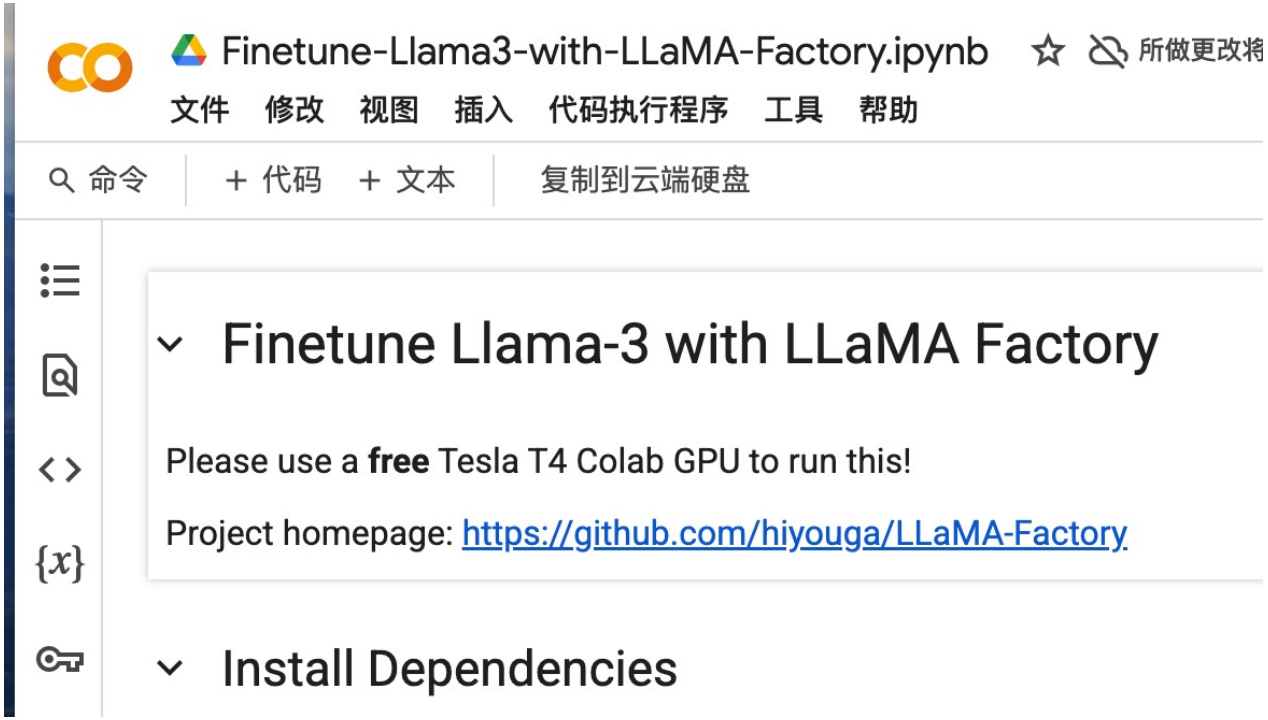
```
CUDA_VISIBLE_DEVICES=0 llamafactory-cli train \
    --stage sft \
    --do_train \
    --model_name_or_path /media/codingma/LLM/llama3/Meta-Llama-3-8B-Instruct \
    --dataset alpaca_gpt4_zh,identity,adgen_local \
    --dataset_dir ./data \
    --template llama3 \
    --finetuning_type lora \可指定微调方式
    --output_dir ./saves/LLaMA3-8B/lora/sft \
```

Web方式

```
llamafactory-cli webchat \
    --model_name_or_path /media/codingma/LLM/llama3/Meta-Llama-3-8B-Instruct \
    --template llama3
```

# 使用LLaMA-Factory进行微调

## Colab演示:




https://colab.research.google.com/drive/1eRTPn37ltBbYsISy9Aw2NuI2Aq5CQrD9?usp=sharing

# What's Next?




## Convolutional Neural Networks

The world of computer vision!



- A new type of neural networks that can understand images
- Computer vision tasks: classification, detection, segmentation, ..

